GOOGLE | ALL SOURCES | TOPIC 7 OF 10 | LESS COMMON TOPIC

Design, Streaming and Scale

18 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	15	2
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Design, Streaming and Scale | 18 questions | tap a title to jump to it

E	1. Implement Batched Undo/Redo Layer	**PracHub**	**2026-02-04**
M	2. Implement a Reusable Locker Allocation System	**PracHub**	**2026-08-14**
M	3. Build Prefix Lookup with a Trie	**PracHub**	**2026-07-03**
M	4. Design a restaurant waitlist system	**PracHub**	**2026-03-01**
M	5. Determine Reporting Relationships	**PracHub**	**2026-02-22**
M	6. Implement Trie Insert and Exact-Word Search	**PracHub**	**2026-02-20**
M	7. Design Tic-Tac-Toe With K Players	**PracHub**	**2026-02-12**
M	8. Restaurant Waitlist: join / delete / find_first_match	**1Point3Acres**	**2026-02-01**
M	9. Find top/bottom-k words in list or stream	**PracHub**	**2026-01-11**
M	10. Design a logger that returns top-K messages	**PracHub**	**2026-01-06**
M	11. Implement iterator merging lists with filtering	**PracHub**	**2026-01-06**
M	12. Design set with O(1) random access	**PracHub**	**2026-01-01**
M	13. Match people to questions using tags and priorities	**PracHub**	**2025-12-27**
M	14. Design server allocation for multi-type nodes	**PracHub**	**2025-09-06**
M	15. Design an elevator control system	**PracHub**	**2025-09-06**
M	16. Hit Counter	**HackTheRounds**	
H	17. Implement LRU Cache.	**Exponent**	
H	18. Implement a function that processes a stream of interval data.	**Exponent**	

STUDY HERE: topic 7, Design, Streaming and Scale

EASY | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 1 of 18

Implement Batched Undo/Redo Layer

POSTED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 158/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #1472 Design Browser History

LeetCode match: LC 1472 Design Browser History (related: undo/redo stacks)

Posted on PracHub: 2026-02-04 (updated 2026-05-06)

Original: https://prachub.com/coding-questions/implement-batched-undo-redo-layer

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch.

Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value afte
r the batch string old_value; // value before the batc
h bool has_old_value; // whether the key existed befor
e the batch }
```

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

QUESTION

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op. Constraints

0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

EXAMPLES

```
Input: [('begin',), ('set', 'color', 'red'),
    ('end',), ('snapshot',), ('undo',),
    ('snapshot',), ('redo',), ('snapshot',)]
Expected Output: [{'color': 'red'}, {}, {'color':
    'red'}]
```

Explanation: A single batch is committed, undone, and redone.

```
Input: [('begin',), ('set', 'a', '1'), ('end',),
    ('begin',), ('set', 'a', '2'), ('set', 'a', '3'),
    ('set', 'b', '9'), ('end',), ('snapshot',),
    ('undo',), ('snapshot',), ('redo',),
    ('snapshot',)]
Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'},
    {'a': '3', 'b': '9'}]
```

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.

Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

APPROACH

Hold the open batch as a map of key -> (oldValue, newValue); endBatch pushes it onto the undo stack and clears the redo stack. undo applies the old values and moves the batch to the redo stack, redo reverses that. Handle an empty batch and repeated sets of one key in a batch (keep the first old value).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what undo does on an empty stack, whether an empty batch counts as an action, whether new edits after an undo clear the redo history (yes, normally), and how nested batches behave.

BRUTE FORCE TO SAY OUT LOUD

Snapshot the whole map after every batch and keep a list of snapshots. Simple but memory heavy.

OPTIMAL IDEA

Hold the open batch as a dict key to (old value, new value), recording the old value only the first time a key is touched in the batch. endBatch pushes it on the undo stack and clears the redo stack. undo pops, restores the old values (deleting keys that did not exist), and pushes onto redo. redo reapplies the new values. O(size of batch) each.

DRY-RUN THESE CASES

Set the same key twice in one batch, set a new key then undo (it must disappear), undo and redo with nothing available, a new batch after an undo (redo cleared), an empty batch.

LIKELY FOLLOW-UPS

Nested batches, bounded history, or undo of individual operations.

PITFALLS

Recording the old value on every set instead of only the first. Forgetting to clear redo on a new batch.

HOW TO THINK ABOUT IT

Undo and redo are two stacks of actions. The question is what an action is: a batch is one action, so record per batch only what is needed to reverse it (the old value of each key touched, and the new one), not a whole copy of the document. A new edit after an undo kills the redo history.

Basis: LeetCode 1472 (Design Browser History) for the stacks idea, plus transactional batching.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
undo, redo = [], []
def commit_batch():
    undo.append(copy.deepcopy(props))
    redo.clear()
def do_undo():
    redo.append(copy.deepcopy(props))
    props.clear()
    props.update(undo.pop())
def do_redo():
    undo.append(copy.deepcopy(props))
    props.clear()
    props.update(redo.pop())
```

TIME	SPACE
O(size of document) per batch	**O(size * batches)**

WHY IT IS SLOW

A one-key change copies the whole document, O(size) per batch and O(size * batches) memory. Storing only the old and new value of each changed key makes each batch O(keys changed).

OPTIMAL SOLUTION · Store only the changed keys per batch

TIME	SPACE
O(keys changed in the batch) per operation	**O(total changed keys)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Limit the history to the last N batches.

Keep the undo stack in a deque with maxlen N so the oldest batch is dropped automatically. Redo is unaffected.

FOLLOW-UP 2 Nested batches.

Keep a stack of open batches and merge a child into its parent on endBatch (keep the parent's old value if the key was already touched). Only the outermost endBatch commits an undoable action.

FOLLOW-UP 3 Undo of individual setProperty calls, not whole batches.

Treat every setProperty outside a batch as a one-key batch (the code does), so undo granularity equals commit granularity.

FOLLOW-UP 4 Limit the history to the last N batches.

Cap the undo stack: when it grows beyond N drop the oldest batch (use a deque with maxlen, or pop(0) as here). The redo stack is unaffected. Memory is then O(N batches).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Batched Undo/Redo Layer
#
# You are implementing a simplified document layer for a
# design tool. The layer stores properties as a
# map<string, string>. Implement a class that supports the
# following operations: - beginBatch(): start a batch of
# edits. - setProperty(key, value): set or overwrite a
# property inside the current batch. - endBatch(): commit
# the batch as one undoable action. - undo(): revert the
# most recently committed batch. - redo(): reapply the
# most recently undone batch. Requirements: 1. A batch may
# update the same key multiple times. 2. undo() must
# restore the exact document state from before that batch.
# 3. redo() must restore the exact document state after
# that batch. 4. If a property did not exist before the
# batch, then undo() should remove that property. 5. If
# undo() has been called and then a new batch is
# committed, the redo history must be cleared. 6. Follow-
# up: make the history memory-efficient.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Limit the history to the last N batches.
# 2. Nested batches.
# 3. Undo of individual setProperty calls, not whole
# batches.
# 4. Limit the history to the last N batches.
#
# INTERVIEW NOTES
# Undo and redo are two stacks of actions. The question is
# what an action is: a batch is one action, so record per
# batch only what is needed to reverse it (the old value
# of each key touched, and the new one), not a whole copy
# of the document. A new edit after an undo kills the redo
# history.
#
# BRUTE FORCE (say it first): Snapshot the whole document
# after each batch
# Push a copy of the map on every commit; undo and redo
# move between snapshots. Obviously correct and a good
# oracle; it copies the whole document for a one-key
# change. Time O(size of document) per batch. Space O(size
# * batches).


from collections import deque
```

```


# HELPERS for the base question
class Change:
    """One key in one batch: value before and after."""

    def __init__(self, key, old, has_old, new):
        self.key, self.old, self.has_old, self.new = (
            key, old, has_old, new)


# BASE QUESTION (optimal): Store only the changed keys per
# batch
# Idea: While a batch is open keep {key: (old, new)} with
# the old value recorded only the first time a key is
# touched. endBatch pushes it on the undo stack and clears
# the redo stack.
# FOLLOW-UP 1: Limit the history to the last N batches.
# Answer: Keep the undo stack in a deque with maxlen N so
# the oldest batch is dropped automatically. Redo is
# unaffected.
# FOLLOW-UP 2: Nested batches.
# Answer: Keep a stack of open batches and merge a child
# into its parent on endBatch (keep the parent's old value
# if the key was already touched). Only the outermost
# endBatch commits an undoable action.
class Layer:
    def __init__(self, history=None):
        self.props = {}
        # FOLLOW-UP 1, 4: history=N keeps the last N
        # batches
        # committed batches; a deque with a maximum length
        # forgets the oldest ones when the history is
        # capped
        self.undo_stack = deque(maxlen=history)
        self.redo_stack = []
        # open batches (a stack, so batches can nest):
        # each is
        # a dict key -> Change, so a key appears only once
        self.open = []

    def begin_batch(self):
        self.open.append({})

    def set_property(self, key, value):
```

```
        if not self.open:
            # outside a batch: a batch of one change
            self.begin_batch()
            self.set_property(key, value)
            self.end_batch()
            return
        batch = self.open[-1]
        if key in batch:
            # touched again: keep the ORIGINAL old value
            # and
            # only update the final value
            batch[key].new = value
        else:
            batch[key] = Change(key, self.props.get(key),
                                key in self.props, value)
        self.props[key] = value

    def end_batch(self):
        batch = self.open.pop()
        if self.open:
            # FOLLOW-UP 2: nested batches fold into the
            # parent
            # a nested batch folds into its parent
            self.merge(self.open[-1], batch)
        elif batch:
            # only the outermost batch is one undoable
            # action;
            # a new action clears the redo history
            self.undo_stack.append(list(batch.values()))
            self.redo_stack.clear()

    @staticmethod
    def merge(parent, child):
        for key, change in child.items():
            if key in parent:
                parent[key].new = change.new
            else:
                parent[key] = change
```

```
    def undo(self):
        if not self.undo_stack:
            return False
        changes = self.undo_stack.pop()
        for c in changes:
            if c.has_old:
```

```
                self.props[c.key] = c.old
            else:
                # the key did not exist before: remove it
                self.props.pop(c.key, None)
        self.redo_stack.append(changes)
        return True

    def redo(self):
        if not self.redo_stack:
            return False
        changes = self.redo_stack.pop()
        for c in changes:
            self.props[c.key] = c.new
        self.undo_stack.append(changes)
        return True


if __name__ == "__main__":
    layer = Layer()
    layer.begin_batch()
    layer.set_property("color", "red")
    layer.set_property("color", "blue")
    layer.set_property("size", "4")
    layer.end_batch()
    assert layer.props == {"color": "blue", "size": "4"}
    # each key is stored once, whatever the number of
    # edits
    assert len(layer.undo_stack[0]) == 2
    layer.undo()
    assert layer.props == {}
    layer.redo()
    assert layer.props == {"color": "blue", "size": "4"}
    layer.begin_batch()
    layer.set_property("color", "green")
    layer.end_batch()
    layer.undo()
    # the old value comes back, not the first value of the
    # batch
    assert layer.props["color"] == "blue"
    # a new batch clears the redo history
    layer.set_property("x", "1")
    assert not layer.redo()
    # nested batches commit as one action
    layer.begin_batch()
    layer.begin_batch()
```

```
    layer.set_property("y", "2")
    layer.end_batch()
    layer.set_property("z", "3")
    layer.end_batch()
    layer.undo()
    assert "y" not in layer.props
    assert "z" not in layer.props
    # history limited to the last 2 batches
    small = Layer(history=2)
    for i in range(4):
        small.set_property("k", str(i))
    assert len(small.undo_stack) == 2
    small.undo()
    small.undo()
    assert not small.undo() and small.props["k"] == "1"
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[list[str]]) -> list[dict[str
, str]]:
SAMPLE INPUT
[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot
',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
```

Test cases

Input	Expected
`[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'color': 'red'}, {}, {'color': 'red'}]`
`[('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'), ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]`

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 2 of 18

Implement a Reusable Locker Allocation System

POSTED 2026-08-14 | 67 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 8.1 30 DAYS 0 PRIORITY 167/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

LeetCode match: LC 1845 Seat Reservation Manager

Posted on PracHub: 2026-08-14 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

QUESTION

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.

Constraints

1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'],
    ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

APPROACH

Min-heap of released locker numbers plus a counter for the next never-used number; store pops the smallest, pickup pushes back. Track the last successful store separately. O(log n) per op.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what store and pickup return, what happens when a customer stores twice, when no locker is free, and whether numbers are reused smallest first.

BRUTE FORCE TO SAY OUT LOUD

Scan lockers 1..n for the first free one on every store: O(n) per operation.

OPTIMAL IDEA

Keep a min-heap of released locker numbers and a counter next_new for lockers never used. store: if the customer already has one return -1; take the heap minimum if the heap is not empty, otherwise next_new if it is at most n, else return -1; remember customer to locker in a dict. pickup: free the customer's locker by pushing it on the heap. O(log n) per operation, and memory depends on used lockers, not n.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

Pickup then store reuses the smallest freed number. Double store by one customer. Store when full. Pickup with no locker.

LIKELY FOLLOW-UPS

n up to a billion (still fine because of the counter), thread safety with a lock, or lockers with sizes.

PITFALLS

Initializing the heap with all n numbers. Not removing the customer record on pickup.

HOW TO THINK ABOUT IT

'Smallest free number' with numbers coming back means a min-heap of returned numbers. Numbers that were never handed out do not need to be stored: keep a counter for the next unused number and use the heap first. That keeps memory proportional to the number of lockers in use, not to n.

Basis: Written; the smallest-available-number pattern (LeetCode 1845 Seat Reservation Manager, 2336 Smallest Number in Infinite Set).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
owner = {}  # customer -> locker
def store(c):
    if c in owner:
        return -1
    taken = set(owner.values())
    for locker in range(1, n + 1):
        if locker not in taken:
            owner[c] = locker
            return locker
    return -1
def pickup(c):
    owner.pop(c, None)
```

TIME	SPACE
O(n) per operation	O(n)

WHY IT IS SLOW

Each store scans up to n lockers, O(n). A min-heap of freed numbers plus a counter for the next unused one makes each operation O(log n).

OPTIMAL SOLUTION · Min-heap of released numbers plus a next-unused counter

TIME	SPACE
O(log n) per operation	O(lockers released and not reused)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 There are 10^9 lockers.

The counter and the heap already make the structure independent of n: nothing is allocated up front, and memory depends only on how many lockers have been released. **Cost:** O(log R) per operation and O(R) memory, where R is the number of lockers released and not yet reused; independent of n.

FOLLOW-UP 2 Many threads call store and pickup.

Protect the heap, the counter and the customer map with one lock so a locker number is never handed out twice (or fine-grained: a lock around the (heap, counter) pair).

FOLLOW-UP 3 Lockers of different sizes.

Keep one heap and counter per size class, and let a customer choose the smallest class that fits. The structure per class is the same.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement a Reusable Locker Allocation
# System
#
# ## Problem A locker bank has lockers numbered from 1
# through n. Process operations in order. store(customer)
# assigns the smallest available locker and returns its
# number, or -1 if that customer already has a locker or
# none is available. pickup(customer) releases and returns
# that customer's locker, or -1 if no locker is assigned.
# last() returns the locker number from the most recent
# successful store, or -1 if there has not been one.
# Return one result per operation. ### Constraints &
# Assumptions - 1 <= n <= 200,000. - There are at most
# 200,000 operations. - Customer IDs are non-empty
# strings. - A released locker becomes immediately
# available for reuse. ### Clarifications - Failed store
# and pickup operations do not change last(). - The
# allocation rule is always the smallest currently free
# locker number. - A customer can store again after a
# successful pickup.
#
# FOLLOW-UPS (answered below, in this order):
# 1. There are 10^9 lockers.
# 2. Many threads call store and pickup.
# 3. Lockers of different sizes.
#
# INTERVIEW NOTES
# 'Smallest free number' with numbers coming back means a
# min-heap of returned numbers. Numbers that were never
# handed out do not need to be stored: keep a counter for
# the next unused number and use the heap first. That
# keeps memory proportional to the number of lockers in
# use, not to n.
#
# BRUTE FORCE (say it first): Scan the lockers for the
# first free one
# Keep an array of owners and scan from 1 for the first
# empty slot on store. Obviously correct, and each store
# can touch every locker. Time O(n) per operation. Space
```

```
# O(n).


import heapq
```

```
# BRUTE FORCE (say it first): Scan the lockers for the
# first free one | time O(n) per operation
class LockersBrute:
    def __init__(self, n):
        self.owner, self.of = [None] * (n + 1), {}

    def store(self, customer):
        if customer in self.of:
            return -1
        for i in range(1, len(self.owner)):
            if self.owner[i] is None:
                self.owner[i], self.of[customer] = (
                    customer,
                    i,
                )
                return i
        return -1

    def pickup(self, customer):
        i = self.of.pop(customer, None)
        if i is None:
            return -1
        self.owner[i] = None
        return i


# BASE QUESTION (optimal): Min-heap of released numbers
# plus a next-unused counter
# Idea: store: if the customer already has a locker return
# -1; take the smallest released number from the heap if
# any, else the next never-used number if it is at most n,
# else -1.
class Lockers:
    def __init__(self, n):
        self.n = n
        # the next never-used locker number
        self.next_new = 1
        # released lockers, smallest on top
        self.free = []
        # customer -> locker number
        self.of = {}
        # locker of the latest successful store
        self.last_store = -1

    def store(self, customer):
```

```
        if customer in self.of:
            return -1
        if self.free:
            # a released locker is always smaller than any
            # new one
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        locker = self.of.pop(customer, None)
        if locker is None:
            return -1
        # it becomes available again at once
        heapq.heappush(self.free, locker)
```

```
        return locker

    def last(self):
        # failed operations never change this
        return self.last_store


# FOLLOW-UP 2: Many threads call store and pickup.
# Answer: Protect the heap, the counter and the customer
# map with one lock so a locker number is never handed out
# twice (or fine-grained: a lock around the (heap,
# counter) pair).
class SafeLockers(Lockers):
    """Many threads: one lock around every operation."""

    def __init__(self, n):
        import threading
        super().__init__(n)
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return super().store(customer)
```

```
    def pickup(self, customer):
        with self.lock:
            return super().pickup(customer)


# HELPERS for the checks below
def process(n, operations):
    box, out = Lockers(n), []
    for op in operations:
        if op[0] == "store":
            out.append(box.store(op[1]))
        elif op[0] == "pickup":
            out.append(box.pickup(op[1]))
        else:
            out.append(box.last())
    return out


if __name__ == "__main__":
    ops = [["store", "a"], ["store", "b"],
           ["pickup", "a"], ["store", "c"], ["last"]]
    assert process(3, ops) == [1, 2, 1, 1, 1]
    # nothing stored yet, and a missing pickup
    empty = [["last"], ["pickup", "x"]]
    assert process(2, empty) == [-1, -1]
    # full, and a customer who already has a locker
    full = [["store", "a"], ["store", "a"],
            ["store", "b"], ["store", "c"]]
    assert process(2, full) == [1, -1, 2, -1]
    # failed operations leave last() alone
    more = full + [["last"]]
    assert process(2, more)[-1] == 2
    # a huge locker bank costs nothing up front
    one = [["store", "a"], ["last"]]
    assert process(10 ** 9, one) == [1, 1]
    s = SafeLockers(2)
    assert s.store("a") == 1 and s.pickup("a") == 1
```

FUNCTION AND TEST CASES

Signature

```
def process_locker_operations(n: int, operations: list[lis
t[str]]) -> list[int]:
```

Sample call

```
process_locker_operations(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
```

Test cases

Input	Expected
`(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])`	`[1, 2, 1, 1, 1]`
`(5, [])`	`[]`

ALSO REPORTED · 1 more

Implement a Reusable Locker Allocation System | PracHub | Technical Screen

https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system-2

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 3 of 18

Build Prefix Lookup with a Trie

POSTED 2026-07-03 | 109 DAYS AGO | LAST 6 MONTHS

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 105/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree) (inferred from overview)

Posted on PracHub: 2026-07-03 (updated 2026-07-04)

Original: https://prachub.com/coding-questions/build-prefix-lookup-with-a-trie

QUESTION

Overview: Practice a Google coding interview problem focused on build prefix lookup with a trie. The prompt emphasizes edge cases, clean implementation, and verifiable test behavior without revealing the solution.

APPROACH

Trie of dict words; walk the prefix, then DFS the subtree to collect completions (optionally store counts at nodes).

HOW TO THINK ABOUT IT

Prefix questions repeat work when each query scans the whole word list. A trie stores each prefix once: walk down by the query, then everything below that node is a completion. Keep counters or a word flag in the nodes to answer follow-ups such as 'how many words start with this' or 'top K completions' without another walk.

Basis: LeetCode 208 (Implement Trie) and 1268 (Search Suggestions System).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(words, prefix):
    found = []
    for word in words:
        if word.startswith(prefix):
            found.append(word)
    return found
```

TIME	SPACE
O(W * L) per query	O(1) extra

WHY IT IS SLOW

BRUTE FORCE

Every query touches every word, O(W * L). A trie walks only the prefix and then the subtree below it.

OPTIMAL SOLUTION · Trie with a DFS below the prefix node

TIME	SPACE
O(L) to walk the prefix + O(size of the answer)	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many words start with this prefix?

Store a counter in each node that is incremented when a word passes through it; the answer is the counter at the end of the prefix walk, O(L) with no DFS.

FOLLOW-UP 2 Return only the top 3 completions (by frequency or alphabetically).

Keep at each node a short list of its best K words (updated on insert), or bound the DFS: visit children in alphabetical order and stop after K words. That is the LeetCode 1268 style answer.

FOLLOW-UP 3 Memory is large (many words with long common prefixes).

A compressed (radix) trie stores whole edge labels instead of single characters; a sorted array with binary search over prefixes uses the least memory for a static dictionary. **Cost:** a radix trie has O(number of words) nodes instead of O(total characters); a sorted array uses O(total characters) with O(L log W) lookups.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Build Prefix Lookup with a Trie
#
# Practice a Google coding interview problem focused on
# build prefix lookup with a trie. The prompt emphasizes
# edge cases, clean implementation, and verifiable test
# behavior without revealing the solution.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many words start with this prefix?
# 2. Return only the top 3 completions (by frequency or
# alphabetically).
# 3. Memory is large (many words with long common
# prefixes).
#
# INTERVIEW NOTES
# Prefix questions repeat work when each query scans the
```

```
# whole word list. A trie stores each prefix once: walk
# down by the query, then everything below that node is a
# completion. Keep counters or a word flag in the nodes to
# answer follow-ups such as 'how many words start with
# this' or 'top K completions' without another walk.
#
# BRUTE FORCE (say it first): Scan every word for the
# prefix
# For each query test word.startswith(prefix) on every
# dictionary word. Correct, but each query pays for the
# whole dictionary. Time O(W * L) per query. Space O(1)
# extra.


# BRUTE FORCE (say it first): Scan every word for the
# prefix | time O(W * L) per query
def completions_brute(words, prefix):
    return sorted(
        w for w in set(words) if w.startswith(prefix)
    )


# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # does a word end here?
        self.is_word = False
        # how many words pass through this node
```

```
        self.count = 0


# BASE QUESTION (optimal): Trie with a DFS below the
# prefix node
# Idea: Insert every word into nested dicts, marking word
# ends. To answer a prefix, walk down one character at a
# time (no match means no completions), then DFS the
# subtree collecting words.
# FOLLOW-UP 1: How many words start with this prefix?
# Answer: Store a counter in each node that is incremented
# when a word passes through it; the answer is the counter
# at the end of the prefix walk, O(L) with no DFS.
class PrefixTrie:
    def __init__(self, words=()):
        self.root = Node()
        for w in words:
            self.insert(w)

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
            node.count += 1
        # a repeated word is only counted as one word
        if not node.is_word:
            node.is_word = True
        else:
            self.undo_count(word)

    def undo_count(self, word):
        node = self.root
        for ch in word:
            node = node.children[ch]
            node.count -= 1

    def walk(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node
```

```
    # FOLLOW-UP 1: how many words start with the prefix
```

```
    def count_prefix(self, prefix):
        # how many words start with the prefix: O(len)
        node = self.walk(prefix)
        if prefix == "":
            return self.root_count()
        return node.count if node else 0

    def root_count(self):
        kids = self.root.children.values()
        return sum(c.count for c in kids)

    # FOLLOW-UP 2: only the top k completions
    def suggest(self, prefix, k=3):
        # first k completions in alphabetical order; the
        # DFS
        # visits children in order and stops after k words
        node = self.walk(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack and len(out) < k:
            n, text = stack.pop()
            if n.is_word:
                out.append(text)
            # reversed so the smallest letter is popped
            # first
            for ch in sorted(n.children, reverse=True):
                stack.append((n.children[ch], text + ch))
        return out

    def completions(self, prefix):
        # every word with the prefix
        return self.suggest(prefix, k=10 ** 9)


if __name__ == "__main__":
    t = PrefixTrie(["apple", "app", "apricot", "banana",
                    "apple"])
    ap = t.completions("ap")
    assert ap == ["app", "apple", "apricot"]
    everything = t.completions("")
    assert everything[-1] == "banana"
    assert t.completions("x") == []
    assert t.count_prefix("ap") == 3
```

```
    assert t.count_prefix("b") == 1
    assert t.count_prefix("zz") == 0
    # the top 2 in alphabetical order
    assert t.suggest("ap", 2) == ["app", "apple"]
    assert t.count_prefix("") == 4
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], queries: list[str]) -> list
[list[str]]:
SAMPLE INPUT
{"words":["car","cat","dog"],"queries":["ca","d"]}
```

Test cases

Input	Expected
{"words":["car","cat","dog"],"queries":["ca","d"]}	[["car","cat"],["dog"]]
{"words":[],"queries":["a"]}	[[]]

ALSO REPORTED · 1 more

Trie Prefix Search / Autocomplete | 1Point3Acres | tech-screen, onsite-coding | asked 2026-07-02 | Medium

https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

Standard implementation: each trie node has a children map and an is_end flag.
Two collection patterns at the prefix: (a) DFS from the prefix node and collect all word-end nodes, (b) store the top-K candidates at every node during insert (Google-preferred for autocomplete).
The standard sorted-suggestions follow-up wants 3 suggestions per prefix-length — handle with sorted children or per-node sorted lists.

Preparation: Write the trie class from memory in under 12 min. Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 4 of 18

★ TOP 75: rank 73 in the Medium and Hard priority list

★ Design a restaurant waitlist system

POSTED 2026-03-01 | 233 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 57.3** **30 DAYS 0** **PRIORITY 49/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #146 LRU Cache

LeetCode match: No direct match; related LC 146 LRU Cache (ordered map)

Posted on PracHub: 2026-03-01 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/design-a-restaurant-waitlist-system

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **c** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**
   - Adds a new party to the end of the waitlist and returns its id.
2. **`cancelParty(partyId) -> bool`**
   - Removes the party from the waitlist if present.
   - Returns **`true`** if removed, otherwise **`false`**.
3. **`seatTable(capacity) -> partyId or null`**
   - Finds and removes the earliest party with **`size <= capacity`**.
   - Returns that party's id, or **`null`** if nobody fits.
4. **`getPosition(partyId) -> int`**
   - Returns how many parties are ahead of this party in the current waitlist order (0-based).
   - If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.

QUESTION

- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

APPROACH

Hash map partyId -> party plus an ordered queue (linked list/OrderedDict) for arrival order; on a free table of size s, seat the earliest waiting party that fits; cancel is O(1) with lazy deletion.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact API (add party, cancel, seat when a table of capacity c frees up), what happens when no party fits, and whether a larger table may seat a smaller party (yes, if size <= c).

BRUTE FORCE TO SAY OUT LOUD

Keep a list in arrival order; on a table of capacity c scan from the front for the first party that fits. O(n) per seating, O(n) per cancel.

OPTIMAL IDEA

Keep id to party in a dict. For efficiency use a segment tree over arrival order storing the minimum party size in each range; the earliest party that fits is the leftmost position with size <= c, found by descending the tree in O(log n). Cancel and seat set that position's size to infinity. The simpler version uses an ordered dict plus lazy deletion and scans, O(n) in the worst case, which is fine to start with and to improve.

DRY-RUN THESE CASES

A table that fits nobody, a party that cancels before being seated, ties on arrival, a table that fits the second party but not the first, duplicate party id.

LIKELY FOLLOW-UPS

Large parties waiting too long (starvation), time-based expiry, or estimating wait time.

PITFALLS

Skipping a party that cancelled but is still in the queue. Choosing the best fit instead of the earliest fit.

HOW TO THINK ABOUT IT

'Earliest arrival whose size fits' is a query on an ordered sequence: find the leftmost position whose value is at most c. A scan does it in O(n); a segment tree that stores the minimum size over each range answers it in O(log n) by walking down to the leftmost child whose range minimum fits. Cancelling or seating simply sets that position to infinity.

HOW TO THINK ABOUT IT

Basis: Written design (compare LeetCode 146 for the ordered-map idea). API assumed (confirm it): add(id, name, size), cancel(id), seat(capacity) seats and returns the earliest-arrived waiting party whose size is at most the capacity, or None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
parties = []  # in arrival order
def join(party):
    parties.append(party)
def seat(capacity):
    for party in parties:
        if party.size <= capacity:
            parties.remove(party)
            return party
    return None
def cancel(party_id):
    for party in parties:
        if party.id == party_id:
            parties.remove(party)
            return
```

TIME	SPACE
O(n) per seat and per cancel	**O(n)**

WHY IT IS SLOW

Both operations walk the list, O(n). A segment tree of the smallest size per range finds the leftmost party that fits in O(log n).

OPTIMAL SOLUTION · Segment tree of minimum size over arrival order

TIME	SPACE
O(log n) per operation	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Parties that wait too long should be seated first (starvation).

Add an aging rule: if the earliest party has waited longer than a threshold and a table of at least its size appears, seat it before anyone else. The segment tree can query 'leftmost size <= c' with c = the table, so order is preserved.

FOLLOW-UP 2 Seat the best-fitting party, not the earliest.

That is a different policy: keep sizes in a sorted structure of (size, arrival) and take the largest size at most the capacity (ties by arrival). A sorted container with bisect gives O(log n) lookups.

FOLLOW-UP 3 Estimate the waiting time of a party.

Track the arrival rate and typical table turnover per capacity; the estimate is the expected time until enough tables of at least that size free up, ahead of the party in the queue.

FOLLOW-UP 4 Seat the best-fitting party (the largest that fits), not the earliest.

Group waiting parties by size in queues (arrival order inside a size) and keep the sizes in a sorted list: the best fit for a table of capacity c is the largest size at most c, found with bisect, then the oldest party of that size. O(log distinct sizes) per seating. Verified against a linear scan.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design a restaurant waitlist system
#
# You are implementing the waitlist system for a
# restaurant. Parties arrive over time, can cancel, and
# get seated when a table becomes available. ## Rules -
# Each party has a unique partyId, a name, and a size
# (number of people). - Parties are seated in **arrival
# order**, but only if they **fit** the available table. -
# When a table of capacity c becomes available, you must
# seat the **earliest-arrived** party whose size <= c. -
# If no party fits, the table remains unused for that
# event. ## Operations to support Design a data structure
# / class that supports the following operations
# efficiently: 1. addParty(name, size) -> partyId - Adds a
# new party to the end of the waitlist and returns its id.
# 2. cancelParty(partyId) -> bool - Removes the party from
# the waitlist if present. - Returns true if removed,
# otherwise false. 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Parties that wait too long should be seated first
# (starvation).
# 2. Seat the best-fitting party, not the earliest.
# 3. Estimate the waiting time of a party.
# 4. Seat the best-fitting party (the largest that fits),
# not the earliest.
#
# INTERVIEW NOTES
# 'Earliest arrival whose size fits' is a query on an
# ordered sequence: find the leftmost position whose value
# is at most c. A scan does it in O(n); a segment tree
# that stores the minimum size over each range answers it
# in O(log n) by walking down to the leftmost child whose
# range minimum fits. Cancelling or seating simply sets
# that position to infinity.
#
# BRUTE FORCE (say it first): Scan the waiting list in
# arrival order
# Keep parties in arrival order in a list. seat scans from
# the front for the first party whose size fits; cancel
# removes by id. Correct and the oracle; every operation
# walks the list. Time O(n) per seat and per cancel. Space
# O(n).
```

```
from bisect import bisect_right, insort
from collections import defaultdict, deque


# BRUTE FORCE (say it first): Scan the waiting list in
# arrival order | time O(n) per seat and per cancel
class WaitlistBrute:
    def __init__(self):
        self.parties = []

    def add(self, pid, name, size):
        self.parties.append((pid, name, size))

    def cancel(self, pid):
        before = len(self.parties)
        self.parties = [
            p for p in self.parties if p[0] != pid
        ]
        return len(self.parties) < before

    def seat(self, capacity):
        for i, p in enumerate(self.parties):
            if p[2] <= capacity:
                del self.parties[i]
                return p[0]
        return None


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Segment tree of minimum size
# over arrival order
# Idea: Each arrival gets the next position. The tree
# stores, for each range, the smallest waiting party size
# (infinity for empty or seated slots).
class Waitlist:
    """Earliest party that fits: a segment tree of the
    smallest size over arrival slots."""

    def __init__(self, capacity=1024):
        self.cap = capacity
        self.tree = [INF] * (2 * capacity)
        self.count = 0
```

```
        self.slot, self.who = {}, {}

    def update(self, i, value):
        i += self.cap
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = min(self.tree[2 * i],
                               self.tree[2 * i + 1])

    def add(self, party_id, name, size):
        self.slot[party_id] = self.count
        self.who[self.count] = party_id
        self.update(self.count, size)
        self.count += 1

    def cancel(self, party_id):
        i = self.slot.get(party_id)
        if i is None or self.tree[i + self.cap] == INF:
            return False
        self.update(i, INF)
        return True

    def seat(self, capacity):
        # no party small enough: nobody is seated
        if self.tree[1] > capacity:
            return None
        i = 1
```

```
        # go left whenever the left half holds a party
        # that fits
        while i < self.cap:
            left = self.tree[2 * i] <= capacity
            i = 2 * i if left else 2 * i + 1
        self.update(i - self.cap, INF)
        return self.who[i - self.cap]


# FOLLOW-UP 2: Seat the best-fitting party, not the
# earliest.
# Answer: That is a different policy: keep sizes in a
# sorted structure of (size, arrival) and take the largest
# size at most the capacity (ties by arrival). A sorted
# container with bisect gives O(log n) lookups.
# FOLLOW-UP 4: Seat the best-fitting party (the largest
# that fits), not the earliest.
```

```
# Answer: Group waiting parties by size in queues (arrival
# order inside a size) and keep the sizes in a sorted
# list: the best fit for a table of capacity c is the
# largest size at most c, found with bisect, then the
# oldest party of that size.
class BestFit:
    """Policy B: the largest party that fits the table."""

    def __init__(self):
        # size -> parties of that size, oldest first
        self.queues = defaultdict(deque)
        self.sizes = []

    def add(self, party_id, size):
        if not self.queues[size]:
            insort(self.sizes, size)
        self.queues[size].append(party_id)

    def seat(self, capacity):
        k = bisect_right(self.sizes, capacity)
        if k == 0:
            return None
        size = self.sizes[k - 1]
        party = self.queues[size].popleft()
        if not self.queues[size]:
            self.sizes.pop(k - 1)
        return party


if __name__ == "__main__":
    w = Waitlist(8)
    w.add("p1", "Ann", 4)
    w.add("p2", "Bob", 2)
    w.add("p3", "Cy", 6)
    # a table for 3: the earliest party that fits is Bob
    assert w.seat(3) == "p2"
    # cancel Ann; a table for 6 now seats Cy
    assert w.cancel("p1") and not w.cancel("p1")
    assert w.seat(6) == "p3"
    assert w.seat(10) is None
    b = BestFit()
    parties = (("a", 2), ("b", 4), ("c", 4), ("d", 6))
    for pid, size in parties:
        b.add(pid, size)
    # a table for 5: the biggest fit is size 4, oldest
```

```
    # first
    assert b.seat(5) == "b"
    assert b.seat(5) == "c"
    assert b.seat(5) == "a"
    assert b.seat(1) is None
```

FUNCTION AND TEST CASES

Test cases

Input	Expected
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('seatTable', 2), ('seatTable', 4)]`	`[1, 2, 2, 1]`
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('addParty', 'C', 3), ('getPosition', 3), ('seatTable', 2), ('getPosition', 3), ('cancelParty', 1), ('getPosition', 3), ('cancelParty', 1), ('seatTable', 2), ('seatTable', 3), ('getPosition', 3)]`	`[1, 2, 3, 2, 2, 1, True, 0, False, None, 3, -1]`

ALSO REPORTED · 1 more

Restaurant Waitlist: join / delete / find_first_match | 1Point3Acres | onsite-coding | asked 2026-02-01 | Single report

https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

Preparation: Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 5 of 18

Determine Reporting Relationships

POSTED 2026-02-22 | 240 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 37.4** **30 DAYS 33** **PRIORITY 63/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #721 Accounts Merge

LeetCode match: No direct match; related LC 721 Accounts Merge (union-find)

Posted on PracHub: 2026-02-22 (updated 2026-04-21)

Original: https://prachub.com/coding-questions/determine-reporting-relationships

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.
- Repeating the same valid relationship is allowed.
- An employee is not considered their own manager.

QUESTION

Constraints

0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles

EXAMPLES

```
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3),
    ("is_manager", 1, 3)])
Expected Output: [True]
```

Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.

```
Input: ([("add_manager", 2, 4), ("is_manager", 1, 4),
    ("add_manager", 1, 2), ("is_manager", 1, 4),
    ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
```

Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4. Hints**`add_peer(a, b)`** means **a** and **b** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **b** through parent pointers.

APPROACH

Manager edges as a directed graph/parent map; peers via union-find or shared-manager sets; queries answer reachability up the manager chain. Keep both structures in sync.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether is_manager means direct manager or any manager up the chain (the example implies the chain), whether peers can be added before their manager is known, and whether input is always consistent (the statement says yes).

BRUTE FORCE TO SAY OUT LOUD

Store all relations and recompute reachability with a graph search on each query. O(V + E) per query.

OPTIMAL IDEA

Keep a Union-Find over employees for peer groups and attach one manager to each group. add_manager(a, b) sets the manager of b's group to a. add_peer(a, b) unions the two groups and, if both had managers, keeps the (necessarily identical) manager. is_manager(a, b): look up the manager of b's group, return true if it is a, otherwise continue from that manager's group up the chain. Each query walks at most the depth of the hierarchy.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

add_manager(m, x) then add_peer(x, y) gives is_manager(m, y) true; add_peer before add_manager; a manager's manager (indirect); an employee with no manager; a query on unknown ids.

LIKELY FOLLOW-UPS

Direct manager only, the closest common manager, or removing relationships.

PITFALLS

Forgetting to merge managers when two groups unite. Treating peers as managers of each other.

HOW TO THINK ABOUT IT

Peers form groups (a peer relation is transitive: same manager as a person who has the same manager as another). A group has at most one direct manager, so store the manager on the group. To answer is_manager(a, b) go from b's group to its manager, then that manager's group to its manager, and so on, looking for a. Union-find makes the groups cheap to merge.

Basis: Written (compare LeetCode 721, Accounts Merge, for the union-find idea). Operations: add_manager(a, b) (a is b's direct manager), add_peer(a, b) (same direct manager), is_manager(a, b) (a is anywhere above b). Inferences must follow from earlier calls.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
managers, peers = [], []  # the stored facts
def is_manager(a, b):
    groups = merge_peers(peers)
    manager_of = {}
    for x, y in managers:  # x manages y
        boss = groups.find(x)
        manager_of[groups.find(y)] = boss
    node = groups.find(b)
    while node in manager_of:
        node = manager_of[node]
        if node == groups.find(a):
            return True
    return False
```

TIME	SPACE
O(F * employees) per query	**O(F)**

WHY IT IS SLOW

Each query redoes the whole inference over all facts. Union-Find for peer groups with a manager stored on each group keeps every update near-constant and a query O(depth).

OPTIMAL SOLUTION · Union-Find for peer groups with a manager stored on each group

TIME	SPACE
O(alpha) per update, O(depth) per query	**O(employees)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the closest common manager of two employees.

Collect the chain of managers above one employee into a set, then climb from the other until you meet it. O(depth).

FOLLOW-UP 2 Facts can be wrong or contradictory.

When uniting groups that already have different managers, the input is inconsistent: raise, or record a conflict instead of silently overwriting. Also guard the upward walk with a visited set so a cycle cannot loop forever.

FOLLOW-UP 3 Remove a relationship.

Union-Find cannot split groups; keep the facts and rebuild lazily, or store the peers as an explicit graph and recompute connected components on demand.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine Reporting Relationships
#
# Build an in-memory organization model that supports
# three operations on employee IDs: 1. add_manager(a, b):
# employee a is the direct manager of employee b. 2.
# add_peer(a, b): employees a and b share the same direct
# manager. 3. is_manager(a, b): return whether employee a
# is a manager of employee b. The answer should account
# for relationships that can be inferred from prior
# operations. For example, if add_manager(m, x) and
# add_peer(x, y) were previously recorded, then
# is_manager(m, y) should return true. You may assume the
# input is logically consistent and does not create
# invalid cycles in the org chart. Explain how you would
# design the data structures and implement these
# operations efficiently.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the closest common manager of two employees.
# 2. Facts can be wrong or contradictory.
# 3. Remove a relationship.
#
# INTERVIEW NOTES
# Peers form groups (a peer relation is transitive: same
# manager as a person who has the same manager as
# another). A group has at most one direct manager, so
# store the manager on the group. To answer is_manager(a,
# b) go from b's group to its manager, then that manager's
# group to its manager, and so on, looking for a. Union-
```

```
# find makes the groups cheap to merge.
#
# BRUTE FORCE (say it first): Recompute all inferences
# from the stored facts on every query
# Keep the list of facts. For a query, repeatedly merge
# peers into groups and attach managers until nothing
# changes, then walk up from b. Correct and the oracle;
# every query redoes the whole inference. Time O(F *
# employees) per query. Space O(F).


# BRUTE FORCE (say it first): Recompute all inferences
# from the stored facts on every query | time O(F *
# employees) per query
class OrgBrute:
    def __init__(self):
        self.mgr, self.peers = [], []

    def add_manager(self, a, b):
        self.mgr.append((a, b))

    def add_peer(self, a, b):
        self.peers.append((a, b))

    def is_manager(self, a, b):
        group = {}
        people = {
            x for p in self.mgr + self.peers for x in p
        }
        for x in people:
            group[x] = {x}
        for x, y in self.peers:
            if group[x] is not group[y]:
                merged = group[x] | group[y]
                for z in merged:
                    group[z] = merged
        boss = {}
        for m, x in self.mgr:
            for z in group[x]:
                boss[id(group[x])] = m
        cur, steps = b, 0
        while (
            cur in group
            and id(group[cur]) in boss
            and steps <= len(people)
        ):
            cur = boss[id(group[cur])]
            if cur == a:
                return True
            steps += 1
        return False


# BASE QUESTION (optimal): Union-Find for peer groups with
# a manager stored on each group
# Idea: Each employee starts in its own group. add_peer
# unions the two groups (and keeps the manager, if either
# has one). add_manager sets the manager of b's group to
# a.
# FOLLOW-UP 1: Return the closest common manager of two
# employees.
```

```
# Answer: Collect the chain of managers above one employee
# into a set, then climb from the other until you meet it.
# O(depth).
class Org:
    def __init__(self):
        # union-find over peers (people with the same
        # manager)
        self.parent = {}
        # peer-group root -> the manager of that group
        self.boss = {}

    def find(self, x):
        self.parent.setdefault(x, x)
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # two groups with different managers contradict
        if ra in self.boss and rb in self.boss \
                and self.boss[ra] != self.boss[rb]:
            # FOLLOW-UP 2: contradictory facts are
            # rejected
            raise ValueError("conflicting managers")
        self.parent[rb] = ra
        # the merged group keeps whichever manager is
        # known
        if rb in self.boss and ra not in self.boss:
            self.boss[ra] = self.boss[rb]

    def add_manager(self, a, b):
        # a is the direct manager of b (and b's peers)
        self.find(a)
        root = self.find(b)
        if root in self.boss and self.boss[root] != a:
            raise ValueError("conflicting managers")
        self.boss[root] = a

    def manager_of(self, x):
        return self.boss.get(self.find(x))
```

```
    def is_manager(self, a, b):
        # climb b's chain of managers; a visited set stops
        # cycles
        seen, cur = set(), b
        while cur is not None and cur not in seen:
            seen.add(cur)
            cur = self.manager_of(cur)
            if cur == a:
                return True
        return False

    # FOLLOW-UP 1: the closest common manager
    def common_manager(self, a, b):
        # the closest manager above both people
        chain, cur = set(), self.manager_of(a)
        while cur is not None and cur not in chain:
            chain.add(cur)
            cur = self.manager_of(cur)
        cur, seen = self.manager_of(b), set()
        while cur is not None and cur not in seen:
            if cur in chain:
                return cur
            seen.add(cur)
            cur = self.manager_of(cur)
        return None


if __name__ == "__main__":
    o = Org()
    o.add_manager("ceo", "vp")
    o.add_manager("vp", "dev")
    o.add_peer("dev", "qa")
    # qa shares dev's manager, so vp manages qa too
    assert o.is_manager("vp", "qa")
    assert o.is_manager("ceo", "qa")
    assert not o.is_manager("qa", "vp")
    assert not o.is_manager("dev", "qa")
    assert o.common_manager("dev", "qa") == "vp"
```

```
    assert o.common_manager("vp", "qa") == "ceo"
    # peers added before the manager is known
    p = Org()
    p.add_peer("a", "b")
    p.add_manager("boss", "a")
    assert p.is_manager("boss", "b")
    # contradictory facts are reported
```

```
    try:
        o.add_manager("ceo", "dev")
        raise AssertionError("should conflict")
    except ValueError:
        pass
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[tuple[str, int, int]]) -> li
st[bool]:
SAMPLE INPUT
([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager"
, 1, 3)])
```

Test cases

Input	Expected
`([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])`	`[True]`
`([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4), ("is_manager", 2, 4), ("is_manager", 4, 2)])`	`[False, True, True, False]`

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 6 of 18

Implement Trie Insert and Exact-Word Search

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 107/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-02-20 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-trie-insert-and-exact-word-search

QUESTION

Problem

Implement a trie that supports `insert(word)` and `search(word)`. Insert adds a lowercase word. Search returns true only when the complete word was previously inserted; a stored word merely sharing the requested prefix is not a match. Process a sequence of operations and return the results of all searches.

Constraints & Assumptions

- There are at most 200,000 operations.
- Every word has length between 1 and 1,000.
- The total number of characters across all operations is at most 1,000,000.
- Words contain lowercase English letters.

Clarifications

- Inserting the same word repeatedly is allowed and has no additional effect.
- A node must distinguish an inserted word ending there from a prefix only.
- Return booleans in the same order as search operations appear.

Examples

```
operations = [insert("apple"), search("apple"), search("app"), insert("app"), search("app")] output = [true, false, true]
```

Hints

```
Represent prefixes Each edge consumes one character and each node represents the prefix along its path.
```

```
Mark complete words Reaching the final character is not enough unless the terminal node records an insertion.
```

APPROACH

Trie nodes with children and an end-of-word flag; search returns true only if the final node is marked as a word.

HOW TO THINK ABOUT IT

A trie needs a flag that marks 'a word ends here'; without it search('app') would be true whenever 'apple' was inserted. Walking a prefix and checking the flag are the two operations: search checks the flag, startsWith does not.

Basis: LeetCode 208, Implement Trie (Prefix Tree).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
words = []
def insert(w):
    words.append(w)
def search(w):
    return w in words
def starts_with(p):
    for word in words:
        if word.startswith(p):
            return True
    return False
```

TIME	SPACE
O(W * L) per operation	O(total characters)

WHY IT IS SLOW

Each call scans every stored word, O(W * L). A trie with an end-of-word flag makes every call O(L).

OPTIMAL SOLUTION · Trie of nodes with an end-of-word flag

TIME	SPACE
O(L) per operation	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support '.' as a wildcard in search (LeetCode 211).

On '.', try every child at that level and recurse; other characters follow the single matching child. Worst case O(26^L) per query on adversarial patterns, so mention the bound.

FOLLOW-UP 2 Delete a word.

Unmark the end flag and, walking back up, remove nodes that have no children and are not word ends. Keep a path stack while walking down so you can prune on the way back. **Cost:** O(L) time and O(L) space for the path stack.

FOLLOW-UP 3 Unicode or large alphabets.

Store children in a dict keyed by character instead of a 26-slot array, as the code does; it handles any alphabet and wastes no space on missing letters. **Cost:** O(L) per operation with dictionary children; memory O(total characters).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Trie Insert and Exact-Word
# Search
#
# ## Problem Implement a trie that supports insert(word)
# and search(word). Insert adds a lowercase word. Search
# returns true only when the complete word was previously
# inserted; a stored word merely sharing the requested
# prefix is not a match. Process a sequence of operations
# and return the results of all searches. ### Constraints
# & Assumptions - There are at most 200,000 operations. -
# Every word has length between 1 and 1,000. - The total
# number of characters across all operations is at most
# 1,000,000. - Words contain lowercase English letters.
# ### Clarifications - Inserting the same word repeatedly
# is allowed and has no additional effect. - A node must
# distinguish an inserted word ending there from a prefix
# only. - Return booleans in the same order as search
# operations appear.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support '.' as a wildcard in search (LeetCode 211).
# 2. Delete a word.
# 3. Unicode or large alphabets.
#
# INTERVIEW NOTES
# A trie needs a flag that marks 'a word ends here';
# without it search('app') would be true whenever 'apple'
# was inserted. Walking a prefix and checking the flag are
# the two operations: search checks the flag, startsWith
# does not.
#
# BRUTE FORCE (say it first): Keep a list of words and
# scan
# Store the words in a list; search compares equality with
# each, startsWith tests the prefix on each. Correct, with
# every call linear in the dictionary size. Time O(W * L)
# per operation. Space O(total characters).


# BRUTE FORCE (say it first): Keep a list of words and
# scan | time O(W * L) per operation
class TrieBrute:
    def __init__(self):
        self.words = []
```

```
    def insert(self, word):
        self.words.append(word)

    def search(self, word):
        return word in self.words

    def startsWith(self, prefix):
        return any(
            w.startswith(prefix) for w in self.words
        )


# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        self.end = False


# BASE QUESTION (optimal): Trie of nodes with an end-of-
# word flag
# Idea: Each node has children by character and a flag.
# insert creates missing nodes and marks the last one.
# search walks the word and requires the flag at the end;
# startsWith only requires that the walk succeeds.
# FOLLOW-UP 1: Support '.' as a wildcard in search
# (LeetCode 211).
# Answer: On '.', try every child at that level and
# recurse; other characters follow the single matching
# child. Worst case O(26^L) per query on adversarial
# patterns, so mention the bound.
class Trie:
    def __init__(self):
        self.root = Node()

    def walk(self, text):
        # follow the letters; None if the path breaks
        node = self.root
        for ch in text:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def insert(self, word):
```

```
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
        node.end = True

    def search(self, word):
        # the whole word must have been inserted
        node = self.walk(word)
        return node is not None and node.end

    def starts_with(self, prefix):
        return self.walk(prefix) is not None

    # FOLLOW-UP 1: '.' matches any one letter
    def search_wild(self, pattern):
        # '.' matches any one letter (LeetCode 211)
        def go(node, i):
            if i == len(pattern):
                return node.end
            ch = pattern[i]
            if ch == ".":
                # try every child at this level
                kids = node.children.values()
                return any(go(c, i + 1) for c in kids)
            child = node.children.get(ch)
            return child is not None and go(child, i + 1)

        return go(self.root, 0)

    # FOLLOW-UP 2: delete a word
```

```
    def delete(self, word):
        # unmark the word, then prune nodes nothing needs
        path = [self.root]
        for ch in word:
            nxt = path[-1].children.get(ch)
            if nxt is None:
                return False
            path.append(nxt)
        if not path[-1].end:
            return False
        path[-1].end = False
        # walk back up removing empty, non-word nodes
        for i in range(len(word), 0, -1):
            node = path[i]
            if node.end or node.children:
```

```
                break
            del path[i - 1].children[word[i - 1]]
        return True


# HELPERS for the checks below
def run(operations):
    # ["insert", w] or ["search", w] -> results of the
    # searches
    trie, out = Trie(), []
    for op, word in operations:
        if op == "insert":
            trie.insert(word)
        else:
            out.append(trie.search(word))
    return out


if __name__ == "__main__":
    ops = [("insert", "apple"), ("search", "apple"),
           ("search", "app"), ("insert", "app"),
           ("search", "app")]
    assert run(ops) == [True, False, True]
    t = Trie()
    t.insert("apple")
    # a shared prefix is not a match for search
    assert not t.search("app") and t.starts_with("app")
    assert not t.starts_with("b")
    t.insert("bad")
    t.insert("dad")
    assert t.search_wild("b.d") and t.search_wild(".ad")
    assert not t.search_wild("b..d")
    assert t.search_wild("...")
    # delete keeps shorter words that share the path
    t.insert("app")
    assert t.delete("apple")
    assert t.search("app") and not t.search("apple")
    assert not t.delete("apple")
    assert t.delete("app") and not t.starts_with("a")
```

FUNCTION AND TEST CASES

Signature

```
def process_trie_operations(operations: list[list[str]]) -
> list[bool]:
```

Sample call

```
process_trie_operations([['insert', 'apple'], ['search', '
apple'], ['search', 'app'], ['insert', 'app'], ['search',
'app']])
```

Test cases

Input	Expected
`([['insert', 'apple'], ['search', 'apple'], ['search', 'app'], ['insert', 'app'], ['search', 'app']],)`	`[True, False, True]`
`([],)`	`[]`

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 7 of 18

Design Tic-Tac-Toe With K Players

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 18.1** **30 DAYS 0** **PRIORITY 153/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

LeetCode match: LC 348 Design Tic-Tac-Toe

Posted on PracHub: 2026-02-12 (updated 2026-05-21)

Original: https://prachub.com/coding-questions/design-tic-tac-toe-with-k-players

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **`n x n`** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.
- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.

QUESTION

- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

Constraints

1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

EXAMPLES

```
Input: (3, 2, [])
Expected Output: []
```

Explanation: No moves are made, so there are no statuses to report.

```
Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2,
    1, 1], [1, 0, 2], [2, 2, 2]])
Expected Output: ['Continue', 'Continue', 'Continue',
    'Continue', '1 won', 'Game Over']
```

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

APPROACH

Per-player row, column, diagonal and anti-diagonal counters; a player wins when any counter reaches n. O(1) per move.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a win needs a full row, column or diagonal (n in a row) or only m in a row, how invalid moves are reported, what happens on a full board, and the API (move(row, col, player) returns winner or none).

BRUTE FORCE TO SAY OUT LOUD

After each move scan the row, column and both diagonals of the board. O(n) per move.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Keep counters per player for each row, each column, the main diagonal and the anti-diagonal. A move increments up to four counters; a player wins when any of their counters reaches n. O(1) per move, O(k * n) memory using a dict of counters. Reject moves on occupied cells.

DRY-RUN THESE CASES

A diagonal win, an anti-diagonal win, a row win by the last player, a draw, a move on an occupied cell, a move after the game has ended.

LIKELY FOLLOW-UPS

Win length m smaller than n (scan four directions from the last move for up to m cells), undo a move, or an unbounded sparse board.

PITFALLS

Counting an occupied cell twice. Not ending the game after a win.

HOW TO THINK ABOUT IT

Do not look at the board after each move. A win needs n marks of the same player on one line, so keep a counter per player and per line (each row, each column, the two diagonals) and bump the at-most-four lines that the move touches. The move wins if any of those counters reaches n.

Basis: LeetCode 348, Design Tic-Tac-Toe, generalized to k players. Assumed rule (confirm it): a player wins by filling a whole row, a whole column or a whole diagonal of the n x n board.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def move(player, r, c):
    board[r][c] = player
    lines = [row(r), column(c)]
    if r == c:
        lines.append(main_diagonal())
    if r + c == n - 1:
        lines.append(anti_diagonal())
    for line in lines:
        if line.count(player) == n:
            return "WIN"
    return "CONTINUE"
```

TIME	SPACE
O(n) per move	O(n^2)

WHY IT IS SLOW

Each move rescans up to four lines, O(n). Keeping a counter per player per row, column and diagonal makes each move O(1).

OPTIMAL SOLUTION · Counters per player and per line

TIME	SPACE
O(1) per move	O(k * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Win with m in a row, not the full line.

Counters per line no longer work, because a line can hold a run anywhere. After each move look along the four directions from that cell, counting same-player neighbours up to m in both directions. O(m) per move.

FOLLOW-UP 2 Undo the last move.

Decrement the same counters you incremented and clear the cell; remember the move history in a stack. The counters make undo as cheap as a move.

FOLLOW-UP 3 The board is huge or unbounded.

Store only the occupied cells in a dict (as TicTacToeM does) and keep the counter-per-line version only if the win condition is a complete line of a bounded board. **Cost:** O(1) per move with line counters on a bounded board, or O(m) per move scanning four directions for an m-in-a-row win; memory O(moves).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design Tic-Tac-Toe With K Players
#
# Design and implement a Tic-Tac-Toe game engine. You are
# given: - k players, each with a unique player ID or
# mark. - An n x n board, initially empty. - Players take
# turns placing their own mark on an empty cell. Unlike
# classic Tic-Tac-Toe, a player wins as soon as they form
# **3 consecutive marks** in a straight line, regardless
# of how large n is. A straight line may be horizontal,
# vertical, diagonal down-right, or diagonal down-left.
# Implement a class or set of functions that supports
# making moves and returns the game status after each
# move, such as: - "Game Over" if a move is attempted
# after the game has ended. - "<player> won" when the
# latest move causes that player to have 3 consecutive
# marks. - A non-terminal status such as "Continue" if the
# game should keep going. Your implementation should
# handle invalid moves, such as placing a mark outside the
# board or on an occupied cell.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Win with m in a row, not the full line.
# 2. Undo the last move.
# 3. The board is huge or unbounded.
#
# INTERVIEW NOTES
```

```
# Do not look at the board after each move. A win needs n
# marks of the same player on one line, so keep a counter
# per player and per line (each row, each column, the two
# diagonals) and bump the at-most-four lines that the move
# touches. The move wins if any of those counters reaches
# n.
#
# BRUTE FORCE (say it first): Scan the lines through the
# last move
# Store the board and, after each move, check the move's
# row, column and (if on one) the diagonals for n equal
# marks. Simple and fast enough for small n, but every
# move rescans up to four lines. Time O(n) per move. Space
# O(n^2).


# BRUTE FORCE (say it first): Scan the lines through the
# last move | time O(n) per move
class TicTacToeBrute:
```

```
    def __init__(self, n):
        self.n, self.board = n, [
            [0] * n for _ in range(n)
        ]

    def move(self, r, c, player):
        if self.board[r][c]:
            return -1
        self.board[r][c] = player
        n, b = self.n, self.board
        if all(
            b[r][j] == player for j in range(n)
        ) or all(b[i][c] == player for i in range(n)):
            return player
        if r == c and all(
            b[i][i] == player for i in range(n)
        ):
            return player
        if r + c == n - 1 and all(
            b[i][n - 1 - i] == player for i in range(n)
        ):
            return player
        return 0


# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


# BASE QUESTION (optimal): Counters per player and per
# line
# Idea: rows[p][r], cols[p][c], diag[p] and anti[p] count
# each player's marks on those lines. A move increments
# the counters for its row, column and, if applicable, the
# diagonals; the player wins when one reaches n.
# FOLLOW-UP 1: Win with m in a row, not the full line.
# Answer: Counters per line no longer work, because a line
# can hold a run anywhere. After each move look along the
# four directions from that cell, counting same-player
# neighbours up to m in both directions. O(m) per move.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        # FOLLOW-UP 3: a huge or unbounded board
        # only occupied cells are stored (works for huge
```

```
        # boards)
        self.cells = {}
        self.history = []
        self.winner = None

    def turn(self):
        # players go 1, 2, ..., k, then 1 again
        return len(self.history) % self.k + 1

    def run_length(self, r, c, dr, dc, player):
        # marks of `player` in a line through (r, c)
        run = 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
            while self.cells.get((a, b)) == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        if self.winner is not None:
            return "Game Over"
        # an invalid move changes nothing
        if not 1 <= player <= self.k:
            return "Invalid move"
        if player != self.turn():
            return "Invalid move"
        if not (0 <= r < self.n and 0 <= c < self.n):
            return "Invalid move"
        if (r, c) in self.cells:
            return "Invalid move"
        self.cells[(r, c)] = player
        self.history.append((r, c))
        # only the four lines through the new mark can win
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc, player)
            # FOLLOW-UP 1: win with `target` in a row
            if run >= self.target:
                self.winner = player
                return f"{player} won"
        return "Continue"

    # FOLLOW-UP 2: undo the last move
    def undo(self):
        # take the last move back (also reopens a finished
```

```
        # game)
        if not self.history:
            return False
        del self.cells[self.history.pop()]
        self.winner = None
        return True


# HELPERS for the checks below
def process(n, k, moves):
    game = TicTacToe(n, k)
    return [game.move(*m) for m in moves]


if __name__ == "__main__":
    moves = [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1],
             [1, 0, 2], [2, 2, 2]]
    assert process(3, 2, moves) == [
        "Continue", "Continue", "Continue", "Continue",
        "1 won", "Game Over"]
    # invalid moves do not use up the turn
    got = process(3, 2, [[2, 0, 0], [1, 5, 5], [1, 0, 0],
                         [2, 0, 0], [3, 1, 1], [2, 1, 1]])
    bad = "Invalid move"
    ok = "Continue"
    assert got == [bad, bad, ok, bad, bad, ok]
    g = TicTacToe(3, 2)
    for m in moves[:5]:
        g.move(*m)
    assert g.winner == 1 and g.undo() and g.winner is None
    assert g.move(1, 0, 2) == "1 won"
    # a huge board costs nothing: only taken cells are
    # stored
    big = TicTacToe(10 ** 9, 2)
```

```
    assert big.move(1, 10 ** 9 - 1, 5) == "Continue"
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, k: int, moves: list[list[int]]) -> li
st[str]:
```

Sample call

```
solution(3, 2, [])
```

Test cases

Input	Expected
`(3, 2, [])`	`[]`
`(5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])`	`['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']`

ALSO REPORTED · 1 more

Tic-Tac-Toe System with K Players and 3-in-a-Row Win | 1Point3Acres | phone-screen, tech-screen | asked 2026-05-18 | Single report

https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

Check four directions through the new mark: horizontal, vertical, main diagonal, anti-diagonal. Count contiguous marks on both sides; win if total reaches 3.

Avoid scanning the whole board after every move unless n is tiny. The local directional check is O(1) for the fixed 3-in-a-row target, or O(target) if generalized.

Clarify invalid moves, turn order, draw state, and whether 3-in-a-row must be exactly 3 or at least 3.

Preparation: Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.

Practice local directional win checks with boundary guards. Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

STUDY HERE: topic 7, Design, Streaming and Scale

Site tags: data-structure, oop-design, medium

MEDIUM | 1Point3Acres | onsite-coding | 8 of 18

Restaurant Waitlist: join / delete / find_first_match

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 216/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

QUESTION

Onsite coding round: design and implement a restaurant waitlist data structure supporting join, delete, and find_first_match(table_size). The find call returns the earliest-joined user whose party size fits the given table.

Implement a class with these methods

join(user) — append user (with their party_size) to the waitlist.
delete(user) — remove user from the waitlist (could be anywhere in the queue).
find_first_match(table_size: int) — return the earliest user still on the waitlist whose party_size <= table_size. Do not remove on find.

Implicit constraints

Order of joins matters (FIFO within those that match).
Users may be deleted out of order, so a plain array/linked list with O(1) join cannot give O(1) delete and find together.

EXAMPLES

```
join(Alice, size=2)
join(Bob, size=4)
join(Carol, size=6)
find_first_match(3) -> Alice   // only Alice fits
find_first_match(5) -> Alice   // Alice still earliest
    matching
delete(Alice)
find_first_match(5) -> Bob
find_first_match(2) -> null     // no one fits
```

APPROACH

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

HOW TO THINK ABOUT IT

Join needs the order of arrival, delete can hit anyone, and find must return the earliest person whose party fits the table. Keep one queue per party size and a sequence number per person: find looks at the front of every queue for sizes up to the table size and takes the smallest sequence number. Deletions are marked and skipped lazily when a queue front is read.

Basis: Written for this question (a doubly linked list plus hash map as in LeetCode 146, LRU Cache, with queues bucketed by party size).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
queue = []  # in arrival order
def join(user):
    queue.append(user)
def delete(user):
    queue.remove(user)
def find_first_match(t):
    for user in queue:
        if user.party_size <= t:
            return user
    return None
```

TIME	SPACE
O(n) per find and per delete	O(n)

WHY IT IS SLOW

Delete and find are linear in the list. One queue per party size with lazy deletion makes join and delete O(1) and find touch only the queue fronts.

OPTIMAL SOLUTION · Queue per party size with lazy deletion

TIME	SPACE
O(1) join and delete, O(S) find (S distinct sizes up to the table size)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What would you cache to speed up repeated finds?

Keep a small structure that answers 'earliest sequence among sizes <= t' directly: a segment tree or Fenwick tree over party size storing the minimum front sequence number, updated when a queue front changes. Find becomes O(log S) instead of O(S).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Restaurant Waitlist: join / delete /
# find_first_match
#
# Onsite coding round: design and implement a restaurant
# waitlist data structure supporting join, delete, and
# find_first_match(table_size). The find call returns the
# earliest-joined user whose party size fits the given
# table. Implement a class with these methods: join(user)
# — append user (with their party_size) to the waitlist.
# delete(user) — remove user from the waitlist (could be
# anywhere in the queue). find_first_match(table_size:
# int) — return the earliest user still on the waitlist
# whose party_size <= table_size. Do not remove on find.
# Implicit constraints Order of joins matters (FIFO within
# those that match). Users may be deleted out of order, so
# a plain array/linked list with O(1) join cannot give
# O(1) delete and find together.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What would you cache to speed up repeated finds?
#
# INTERVIEW NOTES
# Join needs the order of arrival, delete can hit anyone,
# and find must return the earliest person whose party
# fits the table. Keep one queue per party size and a
# sequence number per person: find looks at the front of
# every queue for sizes up to the table size and takes the
# smallest sequence number. Deletions are marked and
# skipped lazily when a queue front is read.
#
# BRUTE FORCE (say it first): Keep one list in join order
# and scan it
# Append on join, remove by value on delete and scan from
# the front on find. Correct and the oracle; delete and
# find are linear. Time O(n) per find and per delete.
# Space O(n).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Keep one list in join order
# and scan it | time O(n) per find and per delete
class WaitlistBrute:
    def __init__(self):
        self.people = []

    def join(self, name, size):
        self.people.append((name, size))

    def delete(self, name):
        self.people = [
            p for p in self.people if p[0] != name
        ]

    def find_first_match(self, table):
        return next(
            (n for n, s in self.people if s <= table),
            None,
        )


# BASE QUESTION (optimal): Queue per party size with lazy
# deletion
# Idea: join stores (sequence number, name) in the queue
# for its size and in a map from name to its record.
# delete just removes the record from the map.
class Waitlist:
    def __init__(self):
        # arrival counter: lower means joined earlier
        self.seq = 0
        # person -> their arrival number (people still
        # waiting)
        self.live = {}
        # party size -> line of (arrival number, person)
        self.queues = defaultdict(deque)

    def join(self, name, size):
        self.seq += 1
        self.live[name] = self.seq
        self.queues[size].append((self.seq, name))

    def delete(self, name):
        # lazy: the entry in its queue is skipped later
        self.live.pop(name, None)

    def front(self, queue):
        # drop deleted people from the front of a queue
        while queue:
            seq, name = queue[0]
            if self.live.get(name) == seq:
                break
            queue.popleft()
        return queue[0] if queue else None

    def find_first_match(self, table):
        best = None
        for size, queue in self.queues.items():
            if size > table:
                continue
            head = self.front(queue)
            # the earliest head among all sizes that fit
            if head and (not best or head[0] < best[0]):
                best = head
        return best[1] if best else None


if __name__ == "__main__":
    w = Waitlist()
    for name, size in [("Alice", 2), ("Bob", 4)]:
        w.join(name, size)
    w.join("Carol", 6)
    assert w.find_first_match(3) == "Alice"
    # Alice is still the earliest who fits 5
    assert w.find_first_match(5) == "Alice"
    # find does not remove anyone
    assert w.find_first_match(5) == "Alice"
    w.delete("Alice")
    assert w.find_first_match(5) == "Bob"
    assert w.find_first_match(1) is None
    # the same size: arrival order decides
    w.join("Dan", 4)
    w.delete("Bob")
    assert w.find_first_match(4) == "Dan"
    # deleting someone who is not there is harmless
    w.delete("Nobody")
```

NOTES FROM CANDIDATES

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

PREPARATION

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 9 of 18

Find top/bottom-k words in list or stream

POSTED 2026-01-11 | 282 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 32.6** **30 DAYS 0** **PRIORITY 117/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words

LeetCode match: LC 692 Top K Frequent Words (+ stream variant)

Posted on PracHub: 2026-01-11 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/find-top-bottom-k-words-in-list-or-stream

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.
Constraints
0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated

QUESTION

EXAMPLES

```
Input: (['i', 'love', 'leetcode', 'i', 'love',
    'coding'], 2)
Expected Output: [['i', 'love'], ['coding',
    'leetcode']]
Explanation: `i` and `love` both appear twice, so
    lexicographical order gives `['i', 'love']`.
    `coding` and `leetcode` both appear once, so
    lexicographical order gives `['coding',
    'leetcode']`.
Input: (['apple', 'banana', 'apple', 'cherry',
    'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'],
    ['cherry', 'date', 'apple']]
```

Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.

Hints
Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).
Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.
Constraints
1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested
Examples
Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]
Expected Output: [['apple', 'banana'], ['banana', 'apple']]
Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.

Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]
Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]
Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.
Hints
Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

APPROACH

List: counts + heaps (min-heap size k for top, max-heap size k for bottom). Stream: keep counts in a map and maintain heaps with lazy updates, or approximate with Space-Saving / Count-Min when memory is bounded.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how ties break, whether top and bottom are over distinct words, what topK(k) returns for k larger than the number of words, and whether memory is bounded for the stream.

BRUTE FORCE TO SAY OUT LOUD

Count words, sort all distinct words by (count, word), take the first and last K. O(d log d).

OPTIMAL IDEA

List version: Counter, then heapq.nlargest and nsmallest with the key (count, word) in the right directions, O(n + d log K). Stream version: ingest increments a dict in O(1); topK and bottomK compute on demand with a heap in O(d log K). If queries are frequent, keep words grouped by count in buckets (a linked list of count buckets like an LFU cache) so topK walks from the highest bucket in O(K). If memory is bounded, the Space-Saving or Count-Min sketch approximates the top K; bottom-K cannot be approximated well, so say so.

DRY-RUN THESE CASES

Fewer words than K, all counts equal (ties decide everything), one word repeated, an empty stream.

LIKELY FOLLOW-UPS

Windowed counts (expire old words), or distributed counting with a merge step.

PITFALLS

Mixing up the direction for the bottom-K heap. Missing a deterministic tie-break.

HOW TO THINK ABOUT IT

Count first; then you only need the K best and K worst, which a heap does without sorting everything. For a stream, ingesting is just a counter increment, and the answer is computed from the counts on demand. If memory is bounded, only heavy hitters (top-K) can be approximated; the rarest words cannot.

Basis: LeetCode 692 (Top K Frequent Words) and its bottom-K counterpart, plus a streaming interface. Ties break alphabetically.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(words, k):
    counts = Counter(words)
    def rank(w):
        return (-counts[w], w)
    ranked = sorted(counts, key=rank)
    most = ranked[:k]
    least = ranked[::-1][:k]
    return most, least
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct words, O(d log d). Two heaps of size K are O(d log k); a stream only needs the counts, and a Space-Saving sketch bounds memory for the top K.

OPTIMAL SOLUTION · Counter plus two heaps of size K

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream is too big to count every distinct word.

Use the Space-Saving (or Misra-Gries) summary with m counters: a word that appears more than n / m times is guaranteed to be tracked, and its stored count is an upper bound. This works for the top K only; the least frequent words cannot be found this way.

FOLLOW-UP 2 topK must be O(k), not O(d log k), because it is called all the time.

Keep words in buckets by count (a linked list of count buckets, like the LFU cache structure); the most frequent words are in the last bucket. An increment moves a word one bucket up in O(1).

FOLLOW-UP 3 Case and punctuation.

Normalise before counting (lowercase, strip punctuation) and say so; the counting and ranking code is unchanged.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find top/bottom-k words in list or stream
#
# You are given words (strings) either as a finite list or
# as an unbounded stream. 1) **List version**: Given an
```

```
# array words and an integer k, return: - the k most
# frequent distinct words, and - the k least frequent
# distinct words. Specify how you break ties (e.g.,
# lexicographically, or any order). 2) **Stream version
# (online)**: Design a data structure that supports: -
# ingest(word): process the next word in the stream -
# topK(k): return the k most frequent distinct words seen
# so far - bottomK(k): return the k least frequent
# distinct words seen so far Discuss time/space complexity
# and how you would implement these operations
# efficiently. 3) **Variant (discuss/design)**: Extend the
# stream design to support returning the **first k**
# and/or **last k** words that are currently **non-
# repeating** (i.e., words with frequency exactly 1)
# according to stream arrival order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream is too big to count every distinct word.
# 2. topK must be O(k), not O(d log k), because it is
# called all the time.
# 3. Case and punctuation.
#
# INTERVIEW NOTES
# Count first; then you only need the K best and K worst,
# which a heap does without sorting everything. For a
# stream, ingesting is just a counter increment, and the
# answer is computed from the counts on demand. If memory
# is bounded, only heavy hitters (top-K) can be
# approximated; the rarest words cannot.
#
# BRUTE FORCE (say it first): Sort all distinct words by
# (count, word)
# Count, sort the distinct words by count and then
# alphabetically, and take the first or last K. Correct
# and the oracle. Time O(d log d). Space O(d).


import heapq
from collections import Counter, defaultdict
```

```
# BRUTE FORCE (say it first): Sort all distinct words by
# (count, word) | time O(d log d)
def top_bottom_sort(words, k):
    c = Counter(words)
    ordered = sorted(c, key=lambda w: (-c[w], w))
    least = sorted(c, key=lambda w: (c[w], w))
    return ordered[:k], least[:k]


# BASE QUESTION (optimal): Counter plus two heaps of size
# K
# Idea: One pass counts the words. heapq.nsmallest with
# key (-count, word) gives the K most frequent and with
# key (count, word) the K least frequent, each in O(d log
# k) without a full sort.
def top_bottom(words, k):
    # ties are broken alphabetically
    c = Counter(words)
    top = heapq.nsmallest(k, c, key=lambda w: (-c[w], w))
    low = heapq.nsmallest(k, c, key=lambda w: (c[w], w))
    bottom = low
    return top, bottom


# FOLLOW-UP 1: The stream is too big to count every
# distinct word.
# Answer: Use the Space-Saving (or Misra-Gries) summary
# with m counters: a word that appears more than n / m
# times is guaranteed to be tracked, and its stored count
# is an upper bound.
class SpaceSaving:
    """Too many distinct words: m counters. A word seen
    over n / m times is always tracked."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, word):
        if word in self.counts:
            self.counts[word] += 1
        elif len(self.counts) < self.m:
            self.counts[word] = 1
        else:
            # replace the smallest counter; the new word
            # inherits its count + 1 (an upper bound)
            low = min(self.counts, key=self.counts.get)
            self.counts[word] = self.counts.pop(low) + 1

    def top_k(self, k):
        return sorted(self.counts, key=lambda w: (
            -self.counts[w], w))[:k]


# FOLLOW-UP 2: topK must be O(k), not O(d log k), because
# it is called all the time.
# Answer: Keep words in buckets by count (a linked list of
# count buckets, like the LFU cache structure); the most
# frequent words are in the last bucket. An increment
# moves a word one bucket up in O(1).
class FreqBuckets:
    """Stream: add(word) is O(1); top_k reads only the
    highest count buckets."""

    def __init__(self):
        self.count = defaultdict(int)
        # count -> words with exactly that count
        self.bucket = defaultdict(dict)
        self.high = 0

    def add(self, word):
        c = self.count[word]
        if c:
            # move the word up one bucket
            del self.bucket[c][word]
            if not self.bucket[c]:
                del self.bucket[c]
        self.count[word] = c + 1
        self.bucket[c + 1][word] = True
        self.high = max(self.high, c + 1)

    def top_k(self, k):
        out = []
        level = self.high
        while level > 0 and len(out) < k:
            # words inside one bucket: alphabetical
            out += sorted(self.bucket.get(level, ()))
            level -= 1
        return out[:k]

    def bottom_k(self, k):
        out = []
        for level in sorted(self.bucket):
            out += sorted(self.bucket[level])
            if len(out) >= k:
                break
        return out[:k]


# FOLLOW-UP 3: Case and punctuation.
# Answer: Normalise before counting (lowercase, strip
# punctuation) and say so; the counting and ranking code
# is unchanged.
def normalise(text):
```

```
    # case and punctuation: clean before counting
    letters = (ch for ch in text.lower() if ch.isalnum())
    return "".join(letters)


if __name__ == "__main__":
    words = "a b a c b a d d".split()
    top, bottom = top_bottom(words, 2)
    assert top == ["a", "b"] and bottom == ["c", "b"]
    f = FreqBuckets()
    for w in words:
        f.add(w)
    assert f.top_k(2) == ["a", "b"]
    assert f.bottom_k(2) == ["c", "b"]
    assert f.top_k(10) == ["a", "b", "d", "c"]
    s = SpaceSaving(2)
    for w in "x x x y z x x":
        s.add(w) if w.strip() else None
    assert s.top_k(1) == ["x"]
    assert normalise("Hello, World!") == "helloworld"
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], k: int) -> list[list[str]]:
```

Sample call

```
solution(['i', 'love', 'leetcode', 'i', 'love', 'coding'],
 2)
```

Test cases

Input	Expected
`(['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)`	`[['i', 'love'], ['coding', 'leetcode']]`
`(['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)`	`[['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]`

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 10 of 18

★ TOP 75: rank 36 in the Medium and Hard priority list

★ Design a logger that returns top-K messages

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 51.2** **30 DAYS 33** **PRIORITY 33/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter, LeetCode #692 Top K Frequent Words

LeetCode match: LC 359 Logger Rate Limiter + LC 692 Top K Frequent Words (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-23)

Original: https://prachub.com/coding-questions/design-a-logger-that-returns-top-k-messages

QUESTION

Overview: This question evaluates skills in designing in-memory data structures and algorithms for frequency counting and interval querying, including extracting top-K elements with a heap, within the Coding & Algorithms domain.

APPROACH

Hash map message -> count (plus a deque of timestamps for interval queries); top-K with a size-k heap, ties broken lexicographically.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy has only the overview, so ask what the API is: log(timestamp, message), top-K over all time or over an interval, whether duplicates within a time gap are suppressed (the LeetCode 359 rate limiter), and how ties break (lexicographically, for LeetCode 692).

BRUTE FORCE TO SAY OUT LOUD

Store every entry in a list and sort or scan on each query. O(n log n) per query.

OPTIMAL IDEA

A dict message to count (and, for interval queries, a dict message to a list of timestamps kept in order so a time window is two binary searches). Top-K uses a heap of size K with the key (count descending, message ascending): heapq.nsmallest(k, items, key=(-count, message)). The rate limiter part is a dict message to last printed time, allowing a message when the gap is at least 10 seconds. O(d log K) per top-K query.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Fewer than K distinct messages, ties on count, the same message repeated inside the rate-limit window, an empty logger.

LIKELY FOLLOW-UPS

Streaming top-K with bounded memory (Space-Saving), or evicting old entries.

PITFALLS

Not specifying the tie rule. Using the full sort when only K items are needed.

HOW TO THINK ABOUT IT

Store enough to answer both questions without scanning everything. A dict from message to its list of timestamps (appended in time order) answers 'how many in [start, end]' with two binary searches, and a heap of size K over (count, message) gives the top K. Say how the logger handles repeated messages and what 'top' means with ties.

Basis: Written (compare LeetCode 359 Logger Rate Limiter and 692 Top K Frequent Words). Assumed API (confirm it): log(timestamp, message); top_k(k, start, end) over an optional time window; ties by message alphabetically.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
logs = []  # (timestamp, message)
def top_k(k, start, end):
    counts = Counter()
    for ts, msg in logs:
        if start <= ts <= end:
            counts[msg] += 1
    def rank(m):
        return (-counts[m], m)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d) per query	**O(n)**

WHY IT IS SLOW

Every query reads the whole log, O(n + d log d). Per-message sorted timestamps with binary search answer a window count in O(log n), and a heap picks the top k.

OPTIMAL SOLUTION · Per-message sorted timestamps plus a heap for the top K

TIME	SPACE
O(1) per log, O(d log n + d log k) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print a message only if it was not printed in the last 10 seconds (LeetCode 359).

Keep a dict message to the time it was last printed and compare: print if the gap is at least 10 (and update the time only when you print). O(1) per call.

FOLLOW-UP 2 Memory must stay bounded.

Expire timestamps older than the largest window you support (drop from the front of each list, delete messages with no timestamps left), or approximate counts with a count-min sketch plus a heap for the heavy hitters. **Cost:** O(1) amortized to drop expired timestamps; memory O(messages inside the window).

FOLLOW-UP 3 Very high top_k frequency of calls.

Maintain counts per message incrementally and a top-K structure (a heap with lazy updates or a sorted structure) so top_k costs O(k) instead of O(d log k); windowed queries still need per-message timestamps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design a logger that returns top-K
# messages
#
# This question evaluates skills in designing in-memory
# data structures and algorithms for frequency counting
# and interval querying, including extracting top-K
# elements with a heap, within the Coding & Algorithms
# domain.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print a message only if it was not printed in the
# last 10 seconds (LeetCode 359).
# 2. Memory must stay bounded.
# 3. Very high top_k frequency of calls.
#
# INTERVIEW NOTES
# Store enough to answer both questions without scanning
# everything. A dict from message to its list of
# timestamps (appended in time order) answers 'how many in
# [start, end]' with two binary searches, and a heap of
# size K over (count, message) gives the top K. Say how
# the logger handles repeated messages and what 'top'
# means with ties.
#
# BRUTE FORCE (say it first): Keep every log entry and
# scan it for each query
# Store all (timestamp, message) pairs; for a query count
# the messages inside the window and sort. Correct and the
# oracle; every query reads the whole log. Time O(n + d
# log d) per query. Space O(n).


from collections import Counter
import heapq
from bisect import bisect_left, bisect_right
from collections import defaultdict
```

```
# BRUTE FORCE (say it first): Keep every log entry and
# scan it for each query | time O(n + d log d) per query
class LoggerBrute:
    def __init__(self):
        self.entries = []

    def log(self, ts, msg):
        self.entries.append((ts, msg))

    def top_k(self, k, start=None, end=None):
        c = Counter(
            m
            for t, m in self.entries
            if (start is None or t >= start)
            and (end is None or t <= end)
        )
        return sorted(c, key=lambda m: (-c[m], m))[:k]


# BASE QUESTION (optimal): Per-message sorted timestamps
# plus a heap for the top K
# Idea: log appends the timestamp to the message's list
# (timestamps arrive in order, so lists stay sorted). A
# window count for one message is bisect_right(end) -
# bisect_left(start).
class Logger:
    def __init__(self):
        # message -> its timestamps, in order of arrival
        self.times = defaultdict(list)

    def log(self, ts, msg):
        self.times[msg].append(ts)

    def top_k(self, k, start=None, end=None):
        # the k most frequent messages in [start, end]
        counts = {}
        for msg, ts in self.times.items():
            lo, hi = 0, len(ts)
            if start is not None:
                lo = bisect_left(ts, start)
            if end is not None:
                hi = bisect_right(ts, end)
            if hi > lo:
                counts[msg] = hi - lo
        # ties are broken alphabetically
        def rank(m):
            return (-counts[m], m)
        return heapq.nsmallest(k, counts, key=rank)

    def expire(self, before):
        # bounded memory: forget timestamps older than
        # `before`
        for msg in list(self.times):
            ts = self.times[msg]
            self.times[msg] = ts[bisect_left(ts, before):]
            if not self.times[msg]:
                del self.times[msg]


# FOLLOW-UP 1: Print a message only if it was not printed
# in the last 10 seconds (LeetCode 359).
# Answer: Keep a dict message to the time it was last
# printed and compare: print if the gap is at least 10
# (and update the time only when you print). O(1) per
# call.
class RateLimiter:
    """LeetCode 359: one print per message per gap."""

    def __init__(self, gap=10):
```

```
        self.gap, self.last = gap, {}

    def should_print(self, ts, msg):
        last = self.last.get(msg)
        if last is not None and ts - last < self.gap:
            return False
        # only a printed message starts a new window
        self.last[msg] = ts
        return True


if __name__ == "__main__":
    log = Logger()
    events = [(1, "a"), (2, "b"), (3, "a"),
              (4, "c"), (5, "a"), (6, "b")]
    for ts, msg in events:
        log.log(ts, msg)
    assert log.top_k(2) == ["a", "b"]
    # only the window [4, 6]: a once, b once, c once
    assert log.top_k(3, 4, 6) == ["a", "b", "c"]
    assert log.top_k(2, 100, 200) == []
    log.expire(4)
    assert log.top_k(5) == ["a", "b", "c"]
    r = RateLimiter()
    times = (1, 2, 11, 12, 21)
    got = [r.should_print(t, "x") for t in times]
    assert got == [True, False, True, False, True]
```

FUNCTION AND TEST CASES

Signature

```
def solution(logs: list[tuple[int, str]], queries: list[li
st[int]]) -> list[list[str]]:
```

Sample call

```
solution([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3
, 2), (1, 10, 3)])
```

Test cases

Input	Expected
`([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3, 2), (1, 10, 3)])`	`[['A', 'B'], ['A', 'B', 'C']]`
`([(1, 'dog'), (2, 'cat'), (2, 'dog'), (4, 'cat'), (5, 'ant')], [(1, 4, 2)])`	`[['cat', 'dog']]`

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 11 of 18

Implement iterator merging lists with filtering

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 155/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #281 Zigzag Iterator

LeetCode match: LC 281 Zigzag Iterator (+ filter/dedupe) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/implement-iterator-merging-lists-with-filtering

QUESTION

Overview: This question evaluates iterator/generator design, sequence merging, stateful filtering and deduplication, checking a candidate's ability to maintain input ordering while excluding blocked IDs and preventing duplicate yields.

APPROACH

Keep one pointer per list and a lookahead 'next valid' value for hasNext/next; skip filtered and duplicate values; or merge with a heap if sorted.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how the lists are merged: round robin (like LeetCode 281 Zigzag Iterator), one after the other, or by sorted value; what is filtered (a blocked set of ids); whether duplicates must be removed across lists; and what next() does when the iterator is exhausted.

BRUTE FORCE TO SAY OUT LOUD

Build one merged list up front with the filter applied. Simple but not lazy and not memory friendly.

OPTIMAL IDEA

A lazy iterator with one pointer per list and a precomputed next valid value (lookahead). hasNext checks the lookahead; next returns it and then advances to the following valid element: skip blocked ids and anything already in a seen set. For sorted merges use a heap of (value, list index, position). Each element is touched once, O(total) overall.

DRY-RUN THESE CASES

Empty lists, all values blocked, duplicates across lists, calling next after the end, a list that becomes empty mid-way.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Make the iterator re-iterable, support remove, or bound memory for the seen set when ids are unbounded (Bloom filter).

PITFALLS

Computing the lookahead lazily at the wrong moment so hasNext skips or repeats a value.

HOW TO THINK ABOUT IT

An iterator does its work lazily: hasNext() must be able to answer without consuming. So keep a lookahead, the next valid value already computed, and advance to the one after it when next() is called. Filtering and de-duplication live in the 'find the next valid value' step so both hasNext and next stay trivial.

Basis: LeetCode 281, Zigzag Iterator, with a blocked-id filter and no duplicate output. Assumed rules (confirm them): lists are read round robin; ids in the blocked set are skipped; each id is yielded at most once.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def merged_iterator(lists, blocked):
    seen = set()
    for x in round_robin(lists):
        if x in blocked or x in seen:
            continue
        seen.add(x)
        yield x
```

TIME	SPACE
O(total)	O(total)

WHY IT IS SLOW

It reads everything up front and keeps it all in memory, so it is not lazy and never works on a long or unbounded source. A lookahead value per iterator computes only the next valid item.

OPTIMAL SOLUTION · Lazy iterator with one pointer per list and a lookahead

TIME	SPACE
O(1) amortized per value	O(number of lists + distinct ids yielded)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The lists are sorted and the output must stay sorted.

Replace the round robin with a k-way merge on a heap (heapq.merge), and drop a value equal to the previous output instead of keeping a seen set: memory O(k).

FOLLOW-UP 2 Read the lists one after another, not interleaved.

Chain the lists with a generator; the same blocked/seen filtering applies. Generators give hasNext-style behaviour via a one-item lookahead.

FOLLOW-UP 3 The seen set grows without bound.

If the input is sorted or ids repeat only close together, remember only the last few ids; otherwise bound memory with a Bloom filter and accept rare false suppressions. **Cost:** O(1) per element with a Bloom filter using O(bits) memory, at the price of rare false suppressions.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement iterator merging lists with
# filtering
#
# This question evaluates iterator/generator design,
# sequence merging, stateful filtering and deduplication,
# checking a candidate's ability to maintain input
# ordering while excluding blocked IDs and preventing
# duplicate yields.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The lists are sorted and the output must stay sorted.
# 2. Read the lists one after another, not interleaved.
# 3. The seen set grows without bound.
#
# INTERVIEW NOTES
# An iterator does its work lazily: hasNext() must be able
# to answer without consuming. So keep a lookahead, the
# next valid value already computed, and advance to the
# one after it when next() is called. Filtering and de-
# duplication live in the 'find the next valid value' step
# so both hasNext and next stay trivial.
#
# BRUTE FORCE (say it first): Build the full merged,
# filtered list first
# Interleave the lists, drop blocked and repeated ids, and
# iterate over the resulting list. Correct and a good
# oracle, but it is not lazy: it reads everything up front
# and holds it all in memory. Time O(total). Space
# O(total).


import heapq
from itertools import chain


# BRUTE FORCE (say it first): Build the full merged,
# filtered list first | time O(total)
def merged_filtered_brute(lists, blocked):
    out, seen = [], set()
    for step in range(
        max((len(l) for l in lists), default=0)
    ):
        for l in lists:
            if (
                step < len(l)
                and l[step] not in blocked
                and l[step] not in seen
            ):
                seen.add(l[step])
                out.append(l[step])
    return out


# BASE QUESTION (optimal): Lazy iterator with one pointer
# per list and a lookahead
# Idea: Keep a pointer per list and a rotation index.
# _advance() scans round robin for the next value that is
# not blocked and not already yielded and stores it as the
# lookahead (or None when everything is exhausted).
class FilteredZigzag:
    """Take one item from each list in turn; skip blocked
    ids and ids already returned. Has hasNext / next."""

    def __init__(self, lists, blocked=()):
        self.lists = lists
        self.blocked = set(blocked)
        self.pos = [0] * len(lists)
        self.turn = 0
        self.seen = set()
        # one item of lookahead makes hasNext easy
        self.ahead = None
        self.advance()

    def advance(self):
        self.ahead = None
        misses = 0
        # stop when every list had nothing left in a full
        # round
        while misses < len(self.lists):
            i = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.pos[i] >= len(self.lists[i]):
                misses += 1
                continue
            value = self.lists[i][self.pos[i]]
            self.pos[i] += 1
            misses = 0
            blocked = value in self.blocked
            skip = blocked or value in self.seen
            if not skip:
                self.ahead = value
                return

    def has_next(self):
        return self.ahead is not None

    def next(self):
        value = self.ahead
        self.seen.add(value)
        self.advance()
        return value


# FOLLOW-UP 1: The lists are sorted and the output must
# stay sorted.
# Answer: Replace the round robin with a k-way merge on a
# heap (heapq.merge), and drop a value equal to the
# previous output instead of keeping a seen set: memory
# O(k).
def merge_sorted(lists, blocked=()):
    # sorted lists, sorted output: a k-way merge; equal
    # values
    # come out together, so remember only the previous one
    blocked, last = set(blocked), object()
    for value in heapq.merge(*lists):
        if value in blocked or value == last:
            continue
        last = value
```

```
        yield value


# FOLLOW-UP 2: Read the lists one after another, not
# interleaved.
# Answer: Chain the lists with a generator; the same
# blocked/seen filtering applies. Generators give hasNext-
# style behaviour via a one-item lookahead.
def read_in_turn(lists, blocked=()):
    # one list after another, with the same filtering
    blocked, seen = set(blocked), set()
    for value in chain.from_iterable(lists):
        if value not in blocked and value not in seen:
            seen.add(value)
            yield value
```

```
if __name__ == "__main__":
    lists = [[1, 2, 3], [2, 4], [5]]
    it = FilteredZigzag(lists, blocked=[4])
    out = []
    while it.has_next():
        out.append(it.next())
    # 1, 2, 5, then 2 (seen), 3, 4 (blocked)
    assert out == [1, 2, 5, 3]
    empty = FilteredZigzag([[], []])
    assert not empty.has_next()
    assert list(merge_sorted([[1, 3, 5], [1, 2, 5]],
                             blocked=[3])) == [1, 2, 5]
    chained = read_in_turn([[1, 2], [2, 3]], [1])
    assert list(chained) == [2, 3]
```

FUNCTION AND TEST CASES

Sample call

```
solution([3, 1, 2, 3], [2, 4, 1, 5], {4})
```

Test cases

Input	Expected
([3, 1, 2, 3], [2, 4, 1, 5], {4})	[3, 1, 2, 5]
([], [1, 1], set())	[1]

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | IN APP LC 380 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 12 of 18

Design set with O(1) random access

POSTED 2026-01-01 | 292 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 46.6** **30 DAYS 0** **PRIORITY 74/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 380 Insert Delete GetRandom O(1)

Posted on PracHub: 2026-01-01 (updated 2026-06-01)

Original: https://prachub.com/coding-questions/design-set-with-o-1-random-access

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **x** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **x** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).
- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.

QUESTION

- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints

1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

EXAMPLES

```
Input: (['add','add','getRandom','delete','getRandom'
    ,'add','getRandom'], [[1],[2],[],[1],[],[2],[]],
    7)
Expected Output: [True, True, 1, True, 2, False, 2]
```

Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

```
Input: (['getRandom','add','add','delete','delete','g
    etRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True,
    None]
```

Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

APPROACH

Array of values + hash map value -> index; remove by swapping with the last element and popping.

HOW TO THINK ABOUT IT

A hash set gives O(1) add and delete but cannot pick a random element in O(1); an array can pick a random element but deleting from the middle is O(n). Combine them: keep the values in an array and a dict from value to array index. To delete, swap the element with the last one, pop the end, and update the moved element's index.

Basis: LeetCode 380, Insert Delete GetRandom O(1) (FancySet). Choice made: getRandom on an empty set raises an error (a defined, loud failure).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []
def add(x):
    if x in items:
        return False
    items.append(x)
    return True
def delete(x):
    if x not in items:
        return False
    items.remove(x)
    return True
def get_random():
    return random.choice(items)
```

TIME	SPACE
O(n) for add and delete, O(1) getRandom	**O(n)**

WHY IT IS SLOW

Membership checks and deletion are O(n). An array plus a dictionary from value to index, deleting by swapping with the last element, makes all three operations O(1).

OPTIMAL SOLUTION · Array of values plus a dict from value to index

TIME	SPACE
O(1) average per operation	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make it thread-safe: one lock or finer-grained locking?

One lock around every operation is simplest and correct (delete touches the array, the dict and a second element's index together, so they must change as one unit). Finer locks do not help much because every operation touches shared structure; use sharding by value hash if contention is the problem.

FOLLOW-UP 2 What should getRandom do on an empty set?

Raise a clear error (as here) or return a sentinel such as None; whichever you pick, document it and make the caller handle it. Returning an arbitrary value hides bugs.

FOLLOW-UP 3 Duplicates allowed (a multiset, LeetCode 381).

Map each value to a set of indices in the array; delete swaps with the last element and fixes both index sets. getRandom stays a uniform pick over array positions, so values are chosen in proportion to their counts.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design set with O(1) random access
#
# Design a data structure ("FancySet") that stores unique
# integers and supports the following operations, each in
# average O(1) time: - add(x): Insert value x if it is not
# already present. Return whether the insert happened. -
# delete(x): Remove value x if it exists. Return whether a
# deletion happened. - getRandom(): Return a uniformly
# random element currently in the set. Requirements /
# follow-ups: - All operations should be average O(1). -
# Handle corner cases such as calling getRandom() on an
# empty set (define and justify behavior). - Discuss how
# you would make this data structure thread-safe. Compare
# using a single lock for the whole set vs finer-grained
# locking (e.g., element-level locks), and explain trade-
# offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make it thread-safe: one lock or finer-grained
# locking?
# 2. What should getRandom do on an empty set?
# 3. Duplicates allowed (a multiset, LeetCode 381).
#
# INTERVIEW NOTES
# A hash set gives O(1) add and delete but cannot pick a
# random element in O(1); an array can pick a random
# element but deleting from the middle is O(n). Combine
# them: keep the values in an array and a dict from value
# to array index. To delete, swap the element with the
# last one, pop the end, and update the moved element's
# index.
#
# BRUTE FORCE (say it first): A plain list
# Store values in a list; add checks for membership with a
# scan and delete removes by search. Correct and the
# oracle; membership and deletion are linear. Time O(n)
# for add and delete, O(1) getRandom. Space O(n).


import random
import threading


# BRUTE FORCE (say it first): A plain list | time O(n) for
# add and delete, O(1) getRandom
```

```
class FancySetBrute:
    def __init__(self):
        self.a = []
```

```
    def add(self, x):
        if x in self.a:
            return False
        self.a.append(x)
        return True

    def delete(self, x):
        if x not in self.a:
            return False
        self.a.remove(x)
        return True

    def getRandom(self):
        if not self.a:
            raise IndexError("empty set")
        return random.choice(self.a)


# BASE QUESTION (optimal): Array of values plus a dict
# from value to index
# Idea: add appends to the array and records the index.
# delete finds the index from the dict, moves the last
# element into that slot, updates its index in the dict,
# pops the array and removes the key.
class FancySet:
    """add, delete and getRandom, each O(1) on average."""

    def __init__(self, rng=None):
        # the values in a list (for random access) and a
        # map
        # from value to its index in that list
        self.items, self.pos = [], {}
        self.rng = rng or random

    def add(self, x):
        if x in self.pos:
            return False
        self.pos[x] = len(self.items)
        self.items.append(x)
        return True
```

```
    def delete(self, x):
        if x not in self.pos:
            return False
        i, last = self.pos[x], self.items[-1]
        # move the last item into the hole, then drop the
        # tail
        self.items[i] = last
        self.pos[last] = i
        self.items.pop()
        del self.pos[x]
        return True

    def get_random(self):
        if not self.items:
            # say what happens on an empty set
            raise IndexError("empty set")
        i = self.rng.randrange(len(self.items))
        return self.items[i]


# FOLLOW-UP 1: Make it thread-safe: one lock or finer-
# grained locking?
# Answer: One lock around every operation is simplest and
# correct (delete touches the array, the dict and a second
# element's index together, so they must change as one
# unit).
class SafeSet(FancySet):
    """One lock: delete changes three things at once."""

    def __init__(self, rng=None):
        super().__init__(rng)
        self.lock = threading.Lock()

    def add(self, x):
        with self.lock:
            return super().add(x)

    def delete(self, x):
        with self.lock:
            return super().delete(x)

    def get_random(self):
        with self.lock:
            return super().get_random()
```

```
# FOLLOW-UP 3: Duplicates allowed (a multiset, LeetCode
# 381).
# Answer: Map each value to a set of indices in the array;
# delete swaps with the last element and fixes both index
# sets. getRandom stays a uniform pick over array
# positions, so values are chosen in proportion to their
# counts.
class RandomizedCollection:
    """Duplicates allowed (LeetCode 381)."""

    def __init__(self):
        self.items = []
        # value -> set of indices where it sits in the
        # list
        self.where = {}

    def insert(self, x):
        first = x not in self.where or not self.where[x]
        spots = self.where.setdefault(x, set())
        spots.add(len(self.items))
        self.items.append(x)
        return first

    def remove(self, x):
        if not self.where.get(x):
            return False
        i = self.where[x].pop()
        end = len(self.items) - 1
        if i != end:
            # move the last item into the hole and fix its
            # index set
            last = self.items[end]
            self.items[i] = last
            self.where[last].discard(end)
            self.where[last].add(i)
        self.items.pop()
        return True

    def get_random(self):
        return random.choice(self.items)


if __name__ == "__main__":
    s = FancySet(random.Random(1))
```

```
    assert s.add(1) and s.add(2) and not s.add(1)
    assert s.delete(1) and not s.delete(1)
    assert s.get_random() == 2
    s.delete(2)
    try:
        s.get_random()
        raise AssertionError("empty set should raise")
    except IndexError:
        pass
    # every value is picked about equally often
    big = FancySet(random.Random(5))
    for v in range(4):
        big.add(v)
```

```
    picks = [big.get_random() for _ in range(4000)]
    for v in range(4):
        assert 850 < picks.count(v) < 1150
    # deleting from the middle keeps the map consistent
    for v in (0, 2):
        big.delete(v)
    assert sorted(big.items) == [1, 3]
    for v, i in big.pos.items():
        assert big.items[i] == v
    safe = SafeSet()
    assert safe.add(7) and safe.get_random() == 7
    rc = RandomizedCollection()
    assert rc.insert(5)
    assert not rc.insert(5)
    assert rc.insert(6)
    assert rc.remove(5) and rc.remove(5)
    assert not rc.remove(5)
    assert rc.items == [6]
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[str], values: list[list[int]
], seed: int) -> str:
```

Sample call

```
solution(['add','add','getRandom','delete','getRandom','ad
d','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
```

Test cases

Input	Expected
`(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)`	`[True, True, 1, True, 2, False, 2]`
`(['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)`	`[None, True, False, False, True, None]`

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | PracHub | Technical Screen | 13 of 18

Match people to questions using tags and priorities

POSTED 2025-12-27 | 297 DAYS AGO | OLDER

STUDY: TALK ALL TIME 5 30 DAYS 0 PRIORITY 172/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

LeetCode match: LC 1820 Maximum Number of Accepted Invitations (bipartite matching) (inferred from overview)

Posted on PracHub: 2025-12-27 (updated 2026-06-04)

Original: https://prachub.com/coding-questions/match-people-to-questions-using-tags-and-priorities

QUESTION

Overview: This question evaluates competencies in bipartite matching and combinatorial optimization, along with practical data-structure design for scalable tag-based assignment and priority-aware selection.

APPROACH

Model as bipartite matching: edges where tags overlap, weights by priority; use Kuhn's/Hopcroft-Karp for max matching, or greedy by priority with an inverted index tag -> candidates.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether matching is one-to-one or a question can take many people, what priority means (higher wins, or a numeric weight to maximize), whether a person needs only one shared tag, and whether it must be maximum cardinality or maximum total priority.

BRUTE FORCE TO SAY OUT LOUD

Try every assignment: exponential. State it and move on.

OPTIMAL IDEA

Build an inverted index tag to candidate questions. If only the number of matches matters it is bipartite matching: Kuhn's augmenting-path algorithm in O(V * E) is enough, Hopcroft-Karp for scale. If priorities are weights use min-cost flow or the Hungarian algorithm. If priority is a strict order, process people by priority and augment, because matchings form a matroid and greedy by priority is optimal.

DRY-RUN THESE CASES

A person with no tag overlap. Two people competing for one question. A chain where the first greedy choice blocks a larger matching (shows why plain greedy fails without augmenting).

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Capacities per question (turns into max flow), online arrivals, or fairness constraints.

PITFALLS

Assuming plain greedy gives a maximum matching. Not building the index and doing all-pairs tag checks.

HOW TO THINK ABOUT IT

'Assign each person at most one question, each question at most one person, allowed pairs given by tags' is maximum bipartite matching. The tag index only builds the edges faster. For maximum count use augmenting paths: try to give a person a question, and if taken ask its current owner to move. If priorities decide who is served, process people in priority order with the same augmenting step.

Basis: LeetCode 1820, Maximum Number of Accepted Invitations (bipartite matching). Assumed rules (confirm them): a person and a question can be matched if they share a tag; each person gets at most one question and each question at most one person; priority decides who is served first when not everyone can be matched.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = 0
def go(person, used, matches):
    global best
    if person == len(people):
        best = max(best, matches)
        return
    go(person + 1, used, matches)  # skip
    for q in range(len(questions)):
        if q in used:
            continue
        if not shares_tag(person, q):
            continue
        go(person + 1, used | {q},
           matches + 1)
```

TIME	SPACE
O((Q + 1)^P)	O(P)

WHY IT IS SLOW

Each person branches over all questions: exponential. Augmenting paths (move an earlier person to another question when needed) solve it in O(people * edges).

OPTIMAL SOLUTION · Augmenting paths (Kuhn's algorithm)

TIME	SPACE
O(P * E)	O(P + Q)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Priorities: higher-priority people must be served first.

Process people in decreasing priority and run the same augmenting step. Matchings of this kind form a matroid, so this greedy choice gives a matched set with the maximum total priority (checked against exhaustive search).

FOLLOW-UP 2 Edges have weights (how good a match is).

Then it is maximum-weight bipartite matching: the Hungarian algorithm in O(n^3), or min-cost max-flow. Kuhn's algorithm only maximises the count.

FOLLOW-UP 3 A question may take several people (capacity c).

Replicate the question c times as separate right-hand nodes, or run max flow with capacities on the question-to-sink edges.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Match people to questions using tags and
# priorities
#
# This question evaluates competencies in bipartite
# matching and combinatorial optimization, along with
# practical data-structure design for scalable tag-based
# assignment and priority-aware selection.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Priorities: higher-priority people must be served
# first.
# 2. Edges have weights (how good a match is).
# 3. A question may take several people (capacity c).
#
# INTERVIEW NOTES
# 'Assign each person at most one question, each question
# at most one person, allowed pairs given by tags' is
# maximum bipartite matching. The tag index only builds
# the edges faster. For maximum count use augmenting
# paths: try to give a person a question, and if taken ask
# its current owner to move. If priorities decide who is
# served, process people in priority order with the same
# augmenting step.
#
# BRUTE FORCE (say it first): Try every assignment
# recursively
# For each person either skip them or give them any unused
# question they share a tag with, and keep the largest
# number of matches. Exact, so it checks the fast
# algorithm, but exponential. Time O((Q + 1)^P). Space
# O(P).


# BRUTE FORCE (say it first): Try every assignment
# recursively | time O((Q + 1)^P)
def max_matching_brute(adj):
    best = 0

    def go(i, used, count):
        nonlocal best
        if i == len(adj):
            best = max(best, count)
            return
        go(i + 1, used, count)
        for q in adj[i]:
            if q not in used:
                go(i + 1, used | {q}, count + 1)

    go(0, frozenset(), 0)
    return best


# BUILD the person-question graph
def build_graph(people, questions):
    # a person can take a question if they share a tag
    # people: [(name, priority, tags)], questions: [(id,
    # tags, cap)]
    adj = {}
    for name, _, tags in people:
        adj[name] = [q for q, qtags, _ in questions
                     if set(tags) & set(qtags)]
    return adj


# BASE QUESTION (optimal): Augmenting paths (Kuhn's
# algorithm)
# Idea: For each person run a DFS that tries each
# acceptable question: if it is free take it, otherwise
# try to move its current owner to another question
# (recursively), using a visited set so no question is
# tried twice in one attempt.
def max_matching(adj):
    # the plain version: adj[p] = list of questions for
    # person p
    owner = {}

    def try_assign(p, seen):
        for q in adj[p]:
            if q in seen:
                continue
            seen.add(q)
            free = q not in owner
            if free or try_assign(owner[q], seen):
                owner[q] = p
                return True
        return False

    people = range(len(adj))
    return sum(try_assign(p, set()) for p in people)


# FOLLOW-UP 1: Priorities: higher-priority people must be
# served first.
# Answer: Process people in decreasing priority and run
# the same augmenting step. Matchings of this kind form a
# matroid, so this greedy choice gives a matched set with
# the maximum total priority (checked against exhaustive
# search).
def assign(people, questions):
    # Kuhn's augmenting paths. People are tried by
    # priority
    # (highest first): such matchings form a matroid, so
    # this
    # greedy order keeps the best possible set of people.
    adj = build_graph(people, questions)
    cap = {q: c for q, _, c in questions}
    # question -> the people currently holding its places
    held = {q: [] for q in cap}

    def try_assign(person, seen):
        for q in adj[person]:
            if q in seen:
```

```
                continue
            seen.add(q)
            # a free place, or someone who can move
            # elsewhere
            if len(held[q]) < cap[q]:
                held[q].append(person)
                return True
            for other in list(held[q]):
                if try_assign(other, seen):
                    held[q].remove(other)
                    held[q].append(person)
                    return True
        return False

    order = sorted(people, key=lambda p: -p[1])
    for name, _, _ in order:
        try_assign(name, set())
    return {p: q for q, ps in held.items() for p in ps}


if __name__ == "__main__":
    assert max_matching([[0, 1], [0], [1, 2]]) == 3
    assert max_matching([[0], [0], [0]]) == 1
```

```
    people = [("ann", 1, {"dp"}),
              ("bob", 5, {"dp", "graph"}),
              ("cy", 3, {"graph"})]
    questions = [("q1", {"dp"}, 1), ("q2", {"graph"}, 1)]
    got = assign(people, questions)
    # bob (priority 5) is served first, and the matching
    # is
    # still the largest possible: ann -> q1, bob or cy ->
    # q2
    assert len(got) == 2
    assert got["bob"] in ("q1", "q2")
    # a question with two places takes two people
    q = [("q1", {"dp"}, 2)]
    both = assign(people[:2], q)
    assert set(both) == {"ann", "bob"}
    # nobody shares a tag: nobody is matched
    nobody = assign([("x", 1, {"a"})], [("q", {"b"}, 1)])
    assert nobody == {}
```

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[list[str]], question_tags: list[
list[str]], priorities: list[int]) -> list[int]:
```

Sample call

```
solution([['python', 'sql'], ['java'], ['ml']], [['sql'],
['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])
```

Test cases

Input	Expected
`([['python', 'sql'], ['java'], ['ml']], [['sql'], ['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])`	`(3, 16)`
`([['a'], ['a']], [['a'], ['a'], ['b']], [1, 10, 100])`	`(2, 11)`

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 14 of 18

Design server allocation for multi-type nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 220/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match; related LC 1912 Design Movie Rental System

Posted on PracHub: 2025-09-06 (updated 2026-08-05)

Original: https://prachub.com/interview-questions/design-server-allocation-for-multi-type-nodes

QUESTION

Design a service to manage a large pool (about 100, 000) of server nodes. Each node can satisfy multiple instance types; clients call getServer(reqNumOfServer, instanceTypes[]) to request servers that collectively meet the requested types, and returnServer(ids[]) to release them. Propose data structures and indexes for fast matching, availability tracking, fragmentation minimization, and scaling. Address concurrency, starvation, and failure handling.

APPROACH

Map instance type -> ordered set of free nodes; allocating a node removes it from every type's set (lazy deletion with an allocated flag); release re-adds it. Pick by the rule in the prompt.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what getServer returns when it cannot be satisfied, whether it must be atomic, what collectively meet the types means (every requested type is covered by at least one chosen server), and the scale (about 100,000 nodes).

BRUTE FORCE TO SAY OUT LOUD

Scan all nodes on every request and test their types. O(N) per call.

OPTIMAL IDEA

Keep, for each instance type, a heap or sorted set of free node ids ordered by how few types the node supports (least flexible first). To serve, pick for each requested type the least flexible free node not already chosen, then fill the remaining count with least flexible nodes that match any requested type. Mark nodes allocated (lazy deletion in the other types' structures). Roll back if the request cannot be completed. Release re-adds the node to each of its types.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Request larger than the free pool, a node shared by two requested types, return then re-request, releasing an id that is not allocated.

LIKELY FOLLOW-UPS

Concurrency (per-type locks or a single allocator thread), starvation (FIFO queue), fragmentation, failure handling (leases and timeouts).

PITFALLS

Allocating a node and forgetting to remove it from the other type indexes. Greedy choices that are not optimal set cover; say so.

HOW TO THINK ABOUT IT

A request mentions a few types, so index the free servers by type and avoid scanning all 100,000. To keep versatile servers available for harder requests, always prefer the server that supports the fewest types. Allocation touches every type index of a server, so use lazy deletion: mark it allocated and let the stale heap entries be skipped when they surface.

Basis: Written design. Assumed contract (confirm it): getServer(count, types) returns `count` free server ids such that every requested type is supported by at least one returned server, or None and no change; releaseServer(ids) frees them. To limit fragmentation the least flexible servers (supporting the fewest types) are used first.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def get_server(count, types):
    chosen = []
    for t in types:
        pool = [s for s in free if t in s.types
                and s not in chosen]
        chosen.append(min(pool, key=flexibility))
    rest = [s for s in free if s not in chosen
            and s.types & set(types)]
    chosen += rest[:count - len(chosen)]
    if len(chosen) < count:
        return None  # not enough
    return chosen
```

TIME	SPACE
O(N * T) per request (N servers)	**O(N)**

WHY IT IS SLOW

Every request scans all N servers, O(N * types). Per-type heaps ordered by flexibility with lazy deletion make a request O((count + types) log N).

OPTIMAL SOLUTION · Per-type heaps ordered by flexibility, with lazy deletion

TIME	SPACE
O((count + types) log N) amortized per request	**O(total type memberships)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many clients call at the same time.

Make allocation atomic: one lock around the allocate step, or shard the pool by type or by id range and route requests to shards. The roll-back-on-failure design means a failed request leaves no trace.

FOLLOW-UP 2 Clients keep asking for rare types and starve.

Use a FIFO queue per request class and hold back versatile servers for waiting rare-type requests, or age requests so an old one outranks new ones. Fragmentation control (least flexible first) already reduces the pressure.

FOLLOW-UP 3 Servers fail while allocated.

Track a lease or heartbeat per allocation; when a server stops responding mark it unavailable, remove it from the heaps (lazy deletion handles stale entries), and tell the client so it can request a replacement.

FOLLOW-UP 4 A client dies without releasing its servers.

Give every allocation a lease (expiry time). A periodic sweep returns servers whose lease has expired to the free pool (the lazy-deletion indexes make a release cheap), and clients renew the lease while they are alive.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design server allocation for multi-type
# nodes
#
# Design a service to manage a large pool (about 100, 000)
# of server nodes. Each node can satisfy multiple instance
# types; clients call getServer(reqNumOfServer,
# instanceTypes[]) to request servers that collectively
# meet the requested types, and returnServer(ids[]) to
# release them. Propose data structures and indexes for
# fast matching, availability tracking, fragmentation
# minimization, and scaling. Address concurrency,
# starvation, and failure handling. Overview: This
# question evaluates understanding of scalable resource-
# allocation and matching systems, encompassing data-
# structure and index design, availability tracking,
# fragmentation minimization, concurrency control, and
# failure handling for large node pools.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many clients call at the same time.
# 2. Clients keep asking for rare types and starve.
# 3. Servers fail while allocated.
# 4. A client dies without releasing its servers.
#
# INTERVIEW NOTES
# A request mentions a few types, so index the free
# servers by type and avoid scanning all 100,000. To keep
# versatile servers available for harder requests, always
# prefer the server that supports the fewest types.
# Allocation touches every type index of a server, so use
# lazy deletion: mark it allocated and let the stale heap
# entries be skipped when they surface.
#
# BRUTE FORCE (say it first): Scan every free server on
# each request
# For each requested type find the best free server by
# scanning all servers, then fill the remaining count the
# same way. Simple and a good oracle; every request
# touches every server. Time O(N * T) per request (N
# servers). Space O(N).


import heapq
from collections import defaultdict
```

```
# BRUTE FORCE (say it first): Scan every free server on
# each request | time O(N * T) per request (N servers)
class ServersBrute:
    def __init__(self, servers):
        self.types = dict(servers)
        self.free = set(servers)

    def getServer(self, count, types):
        chosen = []
        for t in types:
            if any(t in self.types[c] for c in chosen):
                continue
            cands = [
                s
                for s in self.free
                if s not in chosen
                and t in self.types[s]
            ]
            if not cands:
                return None
            chosen.append(
                min(
                    cands,
                    key=lambda s: (
                        len(self.types[s]),
                        s,
                    ),
                )
            )
        extra = sorted(
            (
                s
                for s in self.free
                if s not in chosen
                and self.types[s] & set(types)
            ),
            key=lambda s: (len(self.types[s]), s),
        )
        if (
            len(chosen) > count
            or len(chosen) + len(extra) < count
        ):
            return None
        chosen += extra[: count - len(chosen)]
```

```
        self.free -= set(chosen)
        return chosen

    def releaseServer(self, ids):
        self.free |= set(ids)


# BASE QUESTION (optimal): Per-type heaps ordered by
# flexibility, with lazy deletion
# Idea: For each type keep a heap of (number of supported
# types, id) for the servers that support it.
# FOLLOW-UP 4: A client dies without releasing its
# servers.
# Answer: Give every allocation a lease (expiry time). A
# periodic sweep returns servers whose lease has expired
# to the free pool (the lazy-deletion indexes make a
# release cheap), and clients renew the lease while they
# are alive.
class ServerPool:
    """Servers with several instance types. Each type has
    a heap of (number of types, id): the LEAST flexible
    server goes first, which keeps versatile ones free."""

    def __init__(self, servers):
        # servers: id -> set of instance types
        self.types = {s: frozenset(t)
                      for s, t in servers.items()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        # FOLLOW-UP 4: a dead client's lease expires
        # lease expiry time of every allocated server
        self.lease = {}
        for s, ts in self.types.items():
            for t in ts:
                heap = self.heaps[t]
                heapq.heappush(heap, (len(ts), s))

    def best_for(self, t, skip):
        # the best free server of type t that is not in
        # `skip`;
        # allocated entries are dropped lazily
        heap, held = self.heaps[t], []
        while heap:
            flex, s = heap[0]
            if s in self.allocated:
```

```
                heapq.heappop(heap)
            elif s in skip:
                held.append(heapq.heappop(heap))
            else:
                break
        best = heap[0] if heap else None
        # put back the entries we only looked past
        for item in held:
            heapq.heappush(heap, item)
        return best

    def get_server(self, count, types):
        # pick servers first, commit only if the whole
        # request works
        chosen = []
        for t in types:
            # a server already chosen may cover this type
            if any(t in self.types[c] for c in chosen):
                continue
            top = self.best_for(t, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        # fill up to `count` with the least flexible free
        # servers
        while len(chosen) < count:
            skip = set(chosen)
            found = [self.best_for(t, skip)
```

```
                     for t in types]
            options = [o for o in found if o]
            if not options:
                return None
            chosen.append(min(options)[1])
        # more servers than allowed were needed to cover
        # the types
        if len(chosen) > count:
            return None
        self.allocated |= set(chosen)
        return chosen

    def release(self, ids):
        for s in ids:
            if s in self.allocated:
                self.allocated.discard(s)
                self.lease.pop(s, None)
```

```
                flex = len(self.types[s])
                for t in self.types[s]:
                    heap = self.heaps[t]
                    heapq.heappush(heap, (flex, s))

    def rent(self, count, types, now, ttl):
        # a client that dies must not hold servers forever
        got = self.get_server(count, types)
        for s in got or []:
            self.lease[s] = now + ttl
        return got

    def expire(self, now):
        # return servers whose lease ran out to the free
        # pool
        late = [s for s in self.lease
                if self.lease[s] <= now]
        self.release(late)
        return late


if __name__ == "__main__":
    pool = ServerPool({1: {"gpu"}, 2: {"gpu", "big"},
                       3: {"big"}, 4: {"cpu"}})
    # the least flexible server for each type is used
    # first
    assert pool.get_server(2, ["gpu", "big"]) == [1, 3]
    # the versatile server 2 is still there for a combined
    # need
    assert pool.get_server(1, ["gpu", "big"]) == [2]
    # nothing is left for a gpu request
    assert pool.get_server(1, ["gpu"]) is None
    # a request that cannot fit in `count` servers changes
    # nothing
    assert pool.get_server(1, ["cpu", "gpu"]) is None
    assert pool.get_server(1, ["cpu"]) == [4]
    pool.release([2])
    assert pool.get_server(1, ["gpu"]) == [2]
    # leases: a dead client's servers come back
    p2 = ServerPool({1: {"a"}})
    assert p2.rent(1, ["a"], now=0, ttl=10) == [1]
    assert p2.get_server(1, ["a"]) is None
    assert p2.expire(now=5) == []
    assert p2.expire(now=10) == [1]
    assert p2.get_server(1, ["a"]) == [1]
```

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 15 of 18

Design an elevator control system

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 221/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (OOP / low-level design)

Posted on PracHub: 2025-09-06 (updated 2026-10-07)

Original: https://prachub.com/interview-questions/design-an-elevator-control-system-2

QUESTION

Design the software for a multi-elevator control system in a high-rise building. Specify classes, scheduling algorithms for car assignments, handling peak traffic, fairness, fault tolerance, and safety constraints. Consider real-time constraints and a simulation and testing strategy.

APPROACH

Model Elevator, Request and Controller classes; scheduling by nearest-car or SCAN/LOOK direction-based assignment; queue targets per elevator, handle peak traffic, fairness, fault tolerance and safety; discuss simulation and testing strategy.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how many elevators and floors, whether to design only the control logic or also the hardware, request types (hall call with direction, destination inside), and what to optimize (wait time, fairness, energy).

BRUTE FORCE TO SAY OUT LOUD

A single queue: serve requests first come first served with the nearest idle car. Easy but wasteful.

OPTIMAL IDEA

Classes: Request, Elevator (floor, direction, state, a set of stops), Controller (assigns requests). Use SCAN/LOOK inside each car: keep moving in the current direction serving stops, then reverse. Assign a hall call to the car with the lowest cost, such as distance plus a penalty if it is moving away or is full. Add a step method that advances time so it can be simulated and tested.

DRY-RUN THESE CASES

One request while idle, a request in the opposite direction, two cars equally close, a car out of service with pending stops, the top and bottom floors.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Peak traffic modes, weight limits, fault tolerance (reassign stops on failure), and safety (doors, overload).

PITFALLS

Putting all logic in one class. Not saying how you will test it.

HOW TO THINK ABOUT IT

Separate three things: the data (Request), one car's behaviour (Elevator: floor, direction, stops, door/state) and the policy that decides which car takes a call (Controller). Say the scheduling rule out loud: a car keeps going in its direction serving stops, then reverses (SCAN/LOOK), which avoids the back-and-forth of first-come-first-served. Add a step() so the whole thing can be simulated and tested.

Basis: Written object-oriented design (the classic low-level design question). A SCAN (elevator) scheduling policy per car with a cost-based assignment of hall calls.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
requests = deque()  # arrival order
def run():
    while requests:
        req = requests.popleft()
        car = nearest_idle_car(req.floor)
        car.go_to(req.floor)
        car.go_to(req.destination)
```

TIME	SPACE
O(requests * cars)	O(requests)

WHY IT IS SLOW

Cars zig-zag across the building and requests wait behind far-away ones, so total travel and waiting are high. Serving stops in sweep order (SCAN) and assigning by cost cuts both.

OPTIMAL SOLUTION · Elevator (SCAN) per car and cost-based assignment of calls

TIME	SPACE
O(cars) per call, O(1) per car per step	O(stops)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Peak traffic (morning lobby rush, evening exodus).

Add modes: during the morning rush park idle cars in the lobby and let them go up with a full load; in the evening do the opposite. Zone cars (low floors, high floors) or use destination dispatch (passengers enter their floor first) to group riders by destination.

FOLLOW-UP 2 Fairness and starvation.

SCAN already guarantees every stop is reached within one sweep. For hall calls add an aging term to the cost so a long-waiting call wins over a closer new one.

FOLLOW-UP 3 Safety and fault tolerance.

Keep door and overload as explicit states; a car that fails is marked out of service and its stops are reassigned (the controller does this), and the controller should be restartable from persistent state of cars and pending calls.

FOLLOW-UP 4 Morning rush: idle cars should wait at the lobby.

When a car has no stops, give it a stop at the lobby (or a configured parking floor per car). Peak modes are just policies layered on the same step() and cost-based assignment; the elevator algorithm does not change.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design an elevator control system
#
# Design the software for a multi-elevator control system
# in a high-rise building. Specify classes, scheduling
# algorithms for car assignments, handling peak traffic,
# fairness, fault tolerance, and safety constraints.
# Consider real-time constraints and a simulation and
# testing strategy.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Peak traffic (morning lobby rush, evening exodus).
# 2. Fairness and starvation.
# 3. Safety and fault tolerance.
# 4. Morning rush: idle cars should wait at the lobby.
#
# INTERVIEW NOTES
# Separate three things: the data (Request), one car's
# behaviour (Elevator: floor, direction, stops,
# door/state) and the policy that decides which car takes
# a call (Controller). Say the scheduling rule out loud: a
# car keeps going in its direction serving stops, then
# reverses (SCAN/LOOK), which avoids the back-and-forth of
# first-come-first-served. Add a step() so the whole thing
# can be simulated and tested.
#
# BRUTE FORCE (say it first): First-come-first-served on
# the nearest idle car
# Serve requests in arrival order, each by whichever car
# is closest at that moment, visiting stops in the order
# they were requested. Easy to explain and fair, but cars
# zig-zag across the building. Time O(requests * cars).
# Space O(requests).


# BRUTE FORCE (say it first): First-come-first-served on
# the nearest idle car | time O(requests * cars)
def fcfs_distance(stops, start=0):
    pos, total = start, 0
    for s in stops:
        total += abs(s - pos)
        pos = s
    return total


# HELPERS for the base question
```

```
INF = float("inf")


class Request:
    def __init__(self, floor, direction):
        # direction: +1 up, -1 down
        self.floor, self.direction = floor, direction


# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
class Elevator:
    def __init__(self, car_id, floor=0):
        self.id, self.floor = car_id, floor
        # 0 = idle, +1 = going up, -1 = going down
        self.direction = 0
        self.stops = set()
        self.in_service = True

    def cost(self, req):
        # how bad is it to give this car the request?
        if not self.in_service:
            return INF
        dist = abs(self.floor - req.floor)
        if self.direction == 0:
            return dist
        # on the way: the car is moving toward the floor
        # and
        # the call goes the same way
        gap = req.floor - self.floor
        ahead = gap * self.direction >= 0
        if ahead and req.direction == self.direction:
            return dist
        # otherwise it finishes its sweep first
        span = max(self.stops) - min(self.stops) + 1
        return dist + 2 * span + 10

    def step(self):
        # one tick: serve a stop here, then move one floor
        served = None
        if self.floor in self.stops:
```

```
            self.stops.discard(self.floor)
            served = self.floor
        if not self.stops:
            self.direction = 0
            return served
        up = [s for s in self.stops if s > self.floor]
        down = [s for s in self.stops if s < self.floor]
        # keep the direction while there are stops ahead
        # (SCAN)
        if (self.direction >= 0 and up) or (
                self.direction < 0 and not down):
            self.direction = 1
        else:
            self.direction = -1
        self.floor += self.direction
        return served


# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
```

```
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
# FOLLOW-UP 1: Peak traffic (morning lobby rush, evening
# exodus).
# Answer: Add modes: during the morning rush park idle
# cars in the lobby and let them go up with a full load;
# in the evening do the opposite.
class Controller:
    def __init__(self, cars, lobby=0):
        self.cars, self.lobby = cars, lobby

    def assign(self, req):
        # the cheapest in-service car takes the call
        car = min(self.cars, key=lambda c: c.cost(req))
        if car.cost(req) == INF:
            return None
        car.stops.add(req.floor)
        return car.id

    def tick(self):
        return {c.id: c.step() for c in self.cars}

    # FOLLOW-UP 3: a failed car hands its stops over
```

```
    def fail(self, car_id):
        # a broken car: take it out and hand its stops to
        # others
        car = next(c for c in self.cars if c.id == car_id)
        car.in_service = False
        orphans, car.stops = car.stops, set()
        for floor in orphans:
            self.assign(Request(floor, 0))

    # FOLLOW-UP 1, 4: morning rush: idle cars wait in the
    # lobby
    def park_idle(self):
        # morning rush: idle cars wait in the lobby
        for car in self.cars:
            if car.in_service and not car.stops:
                car.stops.add(self.lobby)


if __name__ == "__main__":
    a, b = Elevator("A", 0), Elevator("B", 10)
    ctl = Controller([a, b])
    # the closest idle car takes the call
    assert ctl.assign(Request(2, 1)) == "A"
    assert ctl.assign(Request(9, -1)) == "B"
    for _ in range(3):
        served = ctl.tick()
    assert a.floor == 2 and served["A"] in (None, 2)
    assert 2 not in a.stops
    # a failed car hands its stops to the other car
    b.stops = {8}
    ctl.fail("B")
    assert not b.in_service and 8 in a.stops
    assert ctl.assign(Request(1, 1)) == "A"
    # nobody in service: the request cannot be taken
    a.in_service = False
    assert ctl.assign(Request(1, 1)) is None
    # idle cars go back to the lobby
    c = Elevator("C", 5)
    Controller([c]).park_idle()
    assert c.stops == {0}
```

STUDY HERE: topic 7, Design, Streaming and Scale

MEDIUM | IN APP LC 362 | DESIGN-STYLE (skippable) | HackTheRounds | Phone Screen | 16 of 18

Hit Counter

STUDY: TALK **ALL TIME 33.2** **30 DAYS 0** **PRIORITY 115/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #362 Design Hit Counter

LeetCode match: LC 362

QUESTION

Design a hit counter that counts the number of hits received in the past 5 minutes (300 seconds). hit(timestamp) records a hit; getHits(timestamp) returns the number of hits in the past 300 seconds.

EXAMPLES

Example 1	
Input	`hit(1), hit(2), hit(3), getHits(4), hit(300), getHits(300), getHits(301)`
Output	`[null, null, null, 3, null, 4, 3]`
Why	`At 301, hit at timestamp 1 expires.`

CONSTRAINTS

- 1 <= timestamp <= 2 * 10^9
- Timestamps are monotonically increasing

APPROACH

Queue/deque of timestamps; evict entries older than 300 seconds

HOW TO THINK ABOUT IT

Only the last 300 seconds matter, so anything older is dead weight: evict from the front of a queue of timestamps. If hits arrive at very high rates, store counts per second in a circular array of 300 buckets so memory does not depend on the number of hits.

Basis: LeetCode 362, Design Hit Counter.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
hits = []
def hit(t):
    hits.append(t)
def get_hits(t):
    return sum(1 for x in hits if x > t - 300)
```

BRUTE FORCE

TIME	SPACE
O(n) per getHits	**O(n)**

WHY IT IS SLOW

Memory grows with every hit forever, and each query scans them all, O(n). A queue that drops timestamps older than 300 seconds (or 300 per-second buckets) keeps both bounded.

OPTIMAL SOLUTION · Queue of timestamps with eviction

TIME	SPACE
O(1) amortized	**O(hits in the window)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Millions of hits per second.

Use 300 buckets (one per second) instead of one entry per hit: hit is O(1), getHits is O(300) = O(1), and memory is constant. A bucket whose stored second is older than the window is reset on reuse.

FOLLOW-UP 2 Timestamps can arrive out of order.

A deque no longer works. Keep a sorted structure (a sorted list with bisect or a balanced tree) and count with two binary searches, or use the buckets, which are order-insensitive within the window.

FOLLOW-UP 3 Thread safety.

Guard the queue with a lock (or use per-thread counters merged on read). The bucket version needs the compare-then-reset of a slot to be atomic. **Cost:** one lock per call, O(1) extra; contention rather than complexity is the concern.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Hit Counter
#
# Design a hit counter that counts the number of hits
# received in the past 5 minutes (300 seconds).
# hit(timestamp) records a hit; getHits(timestamp) returns
# the number of hits in the past 300 seconds.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Millions of hits per second.
# 2. Timestamps can arrive out of order.
# 3. Thread safety.
```

```
#
# INTERVIEW NOTES
# Only the last 300 seconds matter, so anything older is
# dead weight: evict from the front of a queue of
# timestamps. If hits arrive at very high rates, store
# counts per second in a circular array of 300 buckets so
# memory does not depend on the number of hits.
#
# BRUTE FORCE (say it first): Store every hit and count in
# the window
# Append every timestamp to a list and count those with
# timestamp > now - 300. Correct, but memory grows forever
# and each query scans everything. Time O(n) per getHits.
# Space O(n).


import threading
from bisect import bisect_right, insort
from collections import deque


# HELPERS for the brute force
WINDOW = 300   # seconds


def brute(hits, ts):
    return sum(1 for t in hits if ts - WINDOW < t <= ts)


# BASE QUESTION (optimal): Queue of timestamps with
# eviction
# Idea: Keep a deque of timestamps in arrival order.
# getHits pops from the front while the oldest is outside
# the window, and the answer is the deque length. Each
```

```
# timestamp is pushed and popped once.
class QueueCounter:
    """One entry per hit; old entries are dropped when
    asked. Timestamps must not go backwards."""

    def __init__(self):
        self.hits = deque()

    def hit(self, ts):
        self.hits.append(ts)

    def get_hits(self, ts):
        # keep only the hits in (ts - 300, ts]
        while self.hits and self.hits[0] <= ts - WINDOW:
            self.hits.popleft()
        return len(self.hits)


# FOLLOW-UP 1: Millions of hits per second.
# Answer: Use 300 buckets (one per second) instead of one
# entry per hit: hit is O(1), getHits is O(300) = O(1),
# and memory is constant. A bucket whose stored second is
# older than the window is reset on reuse.
class BucketCounter:
    """Follow-up: millions of hits per second. One bucket
    per second of the window: constant memory and time."""

    def __init__(self):
        self.stamp = [0] * WINDOW
        self.count = [0] * WINDOW

    def hit(self, ts):
        slot = ts % WINDOW
        if self.stamp[slot] != ts:
            # the slot still holds an old second: reuse it
            self.stamp[slot], self.count[slot] = ts, 0
        self.count[slot] += 1

    def get_hits(self, ts):
        slots = zip(self.stamp, self.count)
        return sum(c for t, c in slots
                   if 0 <= ts - t < WINDOW)


# FOLLOW-UP 2: Timestamps can arrive out of order.
```

```
# Answer: A deque no longer works. Keep a sorted structure
# (a sorted list with bisect or a balanced tree) and count
# with two binary searches, or use the buckets, which are
# order-insensitive within the window.
class SortedCounter:
    """Follow-up: timestamps can arrive out of order.
    A sorted list and two binary searches count the
    window."""

    def __init__(self):
        self.hits = []

    def hit(self, ts):
        insort(self.hits, ts)

    def get_hits(self, ts):
        newest = bisect_right(self.hits, ts)
        oldest = bisect_right(self.hits, ts - WINDOW)
        return newest - oldest


# FOLLOW-UP 3: Thread safety.
# Answer: Guard the queue with a lock (or use per-thread
# counters merged on read). The bucket version needs the
# compare-then-reset of a slot to be atomic. Cost: one
# lock per call, O(1) extra; contention rather than
# complexity is the concern.
class Locked:
    """Follow-up: thread safety. One lock around every
    call keeps check-then-update steps atomic."""

    def __init__(self, counter):
        self.counter = counter
        self.lock = threading.Lock()

    def hit(self, ts):
        with self.lock:
            self.counter.hit(ts)

    def get_hits(self, ts):
        with self.lock:
            return self.counter.get_hits(ts)


if __name__ == "__main__":
```

```
    kinds = (QueueCounter, BucketCounter, SortedCounter)
    for make in kinds:
        c = make()
        for ts in (1, 2, 3, 300):
            c.hit(ts)
        assert c.get_hits(300) == 4
        # the hit at second 1 is 300 seconds old: it is
        # out
        assert c.get_hits(301) == 3
        assert c.get_hits(599) == 1
        assert c.get_hits(600) == 0
        assert c.get_hits(1000) == 0
    import random
    for make in kinds:
        c, log, now = make(), [], 0
        for _ in range(500):
            now += random.randint(0, 40)
            if random.random() < 0.6:
                c.hit(now)
```

```
            log.append(now)
        else:
            assert c.get_hits(now) == brute(log, now)
# out of order input only works with the sorted
# version
late = SortedCounter()
for ts in (50, 10, 400, 30):
    late.hit(ts)
assert late.get_hits(400) == 1
assert late.get_hits(310) == 2
shared = Locked(BucketCounter())
def spam():
    for _ in range(1000):
        shared.hit(7)

workers = [threading.Thread(target=spam)
           for _ in range(4)]
for w in workers:
    w.start()
for w in workers:
    w.join()
assert shared.get_hits(7) == 4000
```

ALSO REPORTED · 1 more

Design a rolling hit counter API | PracHub | Technical Screen

https://prachub.com/coding-questions/design-a-rolling-hit-counter-api

STUDY HERE: topic 7, Design, Streaming and Scale

HARD | IN APP LC 146 | DESIGN-STYLE (skippable) | Exponent | 17 of 18

★ TOP 75: rank 64 in the Medium and Hard priority list

★ Implement LRU Cache.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 57.3 30 DAYS 0 PRIORITY 48/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: your app's interview approach (LeetCode #146)

LeetCode match: LC 146 LRU Cache

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +2562 interviews

Original: https://www.aced.io/questions/3625/lru-cache

QUESTION

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items—also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n. It should support a set(key, value) method for inserting or updating items and a get(key) method for retrieving items. Can you implement a solution where both of these methods run in O(1) time? Pythonclass LRUCache(n): set(key, value) get(key) Examples To make room for new items, the least recently used item should be removed. An item is 'used' whenever it is set, retrieved, or updated. Pythoncache = LRUCache(2)

Limit of 2 items cache.set('user1',

'Alex') cache.set('user2', 'Brian') cache.set('user3', 'Chris') Here user1 is empty because it was the least recently used and thus removed to make room for user3: Pythoncache.get('user1')

=> None cache.get('user2')

=> 'Brian' cache.get('user3')

=> 'Chris'Hint 1If you're having

trouble getting started, think about which data structure(s) you can use to solve this problem. Which one is best for a key-value store? Which one can help you store items in order of recency?Hint 2You'll need to be able to find and remove the least recent item in O(1) time, so some sort of ordered list is necessary. An array

APPROACH

Hash map + doubly linked list (or OrderedDict): get/put move the node to the front, evict from the tail when over capacity. O(1) per op.

HOW TO THINK ABOUT IT

You need two things at once: find a key quickly (a hash map) and know which key is the oldest use (an ordering you can update in O(1)). A doubly linked list gives O(1) move-to-front and remove-from-back, and each hash map entry points at its list node. Python's OrderedDict is exactly that pair; in an interview be ready to build the linked list by hand.

Basis: LeetCode 146, LRU Cache: get and put in O(1), evicting the least recently used key when over capacity.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []  # (key, value), newest last
def get(k):
    for i, (key, value) in enumerate(items):
        if key == k:
            items.append(items.pop(i))  # newest
            return value
    return -1
def put(k, v):
    for i, (key, _) in enumerate(items):
        if key == k:
            items.pop(i)
            break
    else:
        if len(items) == capacity:
            items.pop(0)  # oldest
    items.append((k, v))
```

TIME	SPACE
O(n) per operation	**O(capacity)**

WHY IT IS SLOW

Every operation scans the list, O(capacity). A hash map to nodes of a doubly linked list gives O(1) lookup and O(1) move-to-front and eviction.

OPTIMAL SOLUTION · Hash map plus doubly linked list (built by hand)

TIME	SPACE
O(1) per operation	**O(capacity)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many threads share the cache.

Wrap get and put in one lock (both reorder the list, so even get is a write). For more throughput shard the cache by key hash, each shard with its own lock and capacity.

FOLLOW-UP 2 Entries expire after a time to live (TTL).

Store an expiry time with each entry, check it on get (a stale entry is treated as missing and removed), and optionally sweep from the old end of the list on put. The recency order and the O(1) operations are unchanged.

FOLLOW-UP 3 Least frequently used (LFU) instead of least recently used.

Keep a frequency per key and buckets of keys per frequency (each bucket an ordered dict) with a pointer to the minimum frequency; get and put move a key to the next bucket in O(1) (LeetCode 460).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement LRU Cache.
#
# Your program is running slowly because it's accessing
# data from disk over and over again. To improve the
# performance, you want to build a simple key-value store
# to cache this data in memory, but you also want to limit
# the amount of memory used. You decide to build a caching
# system that only keeps the N most recently used
# items—also known as a least recently used (LRU) cache.
# Write a class LRUCache(n) that accepts a size limit n.
# It should support a set(key, value) method for inserting
# or updating items and a get(key) method for retrieving
# items. Can you implement a solution where both of these
# methods run in O(1) time? Pythonclass LRUCache(n):
# set(key, value) get(key) Examples To make room for new
# items, the least recently used item should be removed.
# An item is 'used' whenever it is set, retrieved, or
# updated.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many threads share the cache.
# 2. Entries expire after a time to live (TTL).
# 3. Least frequently used (LFU) instead of least recently
# used.
#
# INTERVIEW NOTES
# You need two things at once: find a key quickly (a hash
# map) and know which key is the oldest use (an ordering
# you can update in O(1)). A doubly linked list gives O(1)
# move-to-front and remove-from-back, and each hash map
# entry points at its list node. Python's OrderedDict is
# exactly that pair; in an interview be ready to build the
# linked list by hand.
#
# BRUTE FORCE (say it first): A list of keys ordered by
# recency
# Keep a list of (key, value) with the most recent at the
# end. get and put search the list and move the entry.
# Obviously correct and the oracle; every operation is a
# linear scan. Time O(n) per operation. Space O(capacity).


import threading
import time
from collections import OrderedDict, defaultdict
```

```
# BRUTE FORCE (say it first): A list of keys ordered by
# recency | time O(n) per operation
class LRUBrute:
    def __init__(self, capacity):
        self.cap, self.items = capacity, []

    def get(self, key):
        for i, (k, v) in enumerate(self.items):
            if k == key:
                self.items.append(self.items.pop(i))
                return v
        return -1

    def put(self, key, value):
        for i, (k, _) in enumerate(self.items):
            if k == key:
                self.items.pop(i)
                break
        else:
            if len(self.items) == self.cap:
                self.items.pop(0)
        self.items.append((key, value))


# HELPERS for the base question
class Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=None, value=None):
        self.key, self.value = key, value
        self.prev = self.next = None


# BASE QUESTION (optimal): Hash map plus doubly linked
# list (built by hand)
# Idea: Each key maps to a node of a doubly linked list
# ordered by recency, with sentinel head and tail nodes so
# no operation has to special-case an empty list.
class LRUCache:
    """get and put in O(1): a hash map plus a doubly
    linked list, most recently used item at the front."""

    def __init__(self, capacity):
```

```
        self.cap, self.map = capacity, {}
        # sentinel nodes mean no special cases for the
        # ends
        self.head, self.tail = Node(), Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def unlink(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def push_front(self, node):
        node.next, node.prev = self.head.next, self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        node = self.map.get(key)
        if node is None:
            return -1
        # reading makes it the most recently used
        self.unlink(node)
        self.push_front(node)
        return node.value

    def put(self, key, value):
        node = self.map.get(key)
        if node:
```

```
            node.value = value
            self.unlink(node)
            self.push_front(node)
            return
        if len(self.map) == self.cap:
            # full: evict the least recently used (at the
            # back)
            oldest = self.tail.prev
            self.unlink(oldest)
            del self.map[oldest.key]
        node = Node(key, value)
        self.map[key] = node
        self.push_front(node)


# FOLLOW-UP 1: Many threads share the cache.
# Answer: Wrap get and put in one lock (both reorder the
```

```
# list, so even get is a write). For more throughput shard
# the cache by key hash, each shard with its own lock and
# capacity.
class SafeLRU(LRUCache):
    """Many threads: even get edits the list: one lock."""

    def __init__(self, capacity):
        super().__init__(capacity)
        self.lock = threading.Lock()

    def get(self, key):
        with self.lock:
            return super().get(key)

    def put(self, key, value):
        with self.lock:
            super().put(key, value)


# FOLLOW-UP 2: Entries expire after a time to live (TTL).
# Answer: Store an expiry time with each entry, check it
# on get (a stale entry is treated as missing and
# removed), and optionally sweep from the old end of the
# list on put. The recency order and the O(1) operations
# are unchanged.
class TTLCache(LRUCache):
    """Entries also expire after ttl seconds."""

    def __init__(self, capacity, ttl,
                 clock=time.monotonic):
        super().__init__(capacity)
        self.ttl, self.clock, self.expiry = ttl, clock, {}

    def get(self, key):
        # a stale entry counts as missing and is removed
        stale = key in self.map and (
            self.expiry[key] <= self.clock())
        if stale:
            self.unlink(self.map.pop(key))
            del self.expiry[key]
        return super().get(key)

    def put(self, key, value):
        super().put(key, value)
        self.expiry[key] = self.clock() + self.ttl
```

```
        # drop expiry records of evicted keys
        for k in list(self.expiry):
            if k not in self.map:
                del self.expiry[k]


# FOLLOW-UP 3: Least frequently used (LFU) instead of
# least recently used.
# Answer: Keep a frequency per key and buckets of keys per
```

```
# frequency (each bucket an ordered dict) with a pointer
# to the minimum frequency; get and put move a key to the
# next bucket in O(1) (LeetCode 460).
class LFUCache:
    """Least FREQUENTLY used (LeetCode 460), O(1)."""

    def __init__(self, capacity):
        self.cap = capacity
        self.value, self.freq = {}, {}
        # frequency -> keys in that bucket, oldest first
        self.bucket = defaultdict(OrderedDict)
        self.low = 0

    def touch(self, key):
        f = self.freq[key]
        del self.bucket[f][key]
        # if this was the lowest bucket and it is empty,
        # move up
        if not self.bucket[f] and self.low == f:
            self.low += 1
        self.freq[key] = f + 1
        self.bucket[f + 1][key] = None

    def get(self, key):
        if key not in self.value:
            return -1
        self.touch(key)
        return self.value[key]

    def put(self, key, value):
        if self.cap == 0:
            return
        if key in self.value:
            self.value[key] = value
            self.touch(key)
            return
```

```
        if len(self.value) == self.cap:
            # evict the oldest key of the lowest frequency
            bucket = self.bucket[self.low]
            old, _ = bucket.popitem(last=False)
            del self.value[old], self.freq[old]
        self.value[key], self.freq[key] = value, 1
        self.bucket[1][key] = None
        self.low = 1


if __name__ == "__main__":
    c = LRUCache(2)
    c.put(1, 1)
    c.put(2, 2)
    assert c.get(1) == 1
    # 2 is now the least recently used, so 3 evicts it
    c.put(3, 3)
    assert c.get(2) == -1 and c.get(3) == 3
    c.put(1, 10)
    assert c.get(1) == 10
    s = SafeLRU(1)
    s.put("a", 1)
    s.put("b", 2)
    assert s.get("a") == -1 and s.get("b") == 2
    now = [0]
    t = TTLCache(2, ttl=10, clock=lambda: now[0])
    t.put("x", 1)
    assert t.get("x") == 1
    now[0] = 11
    assert t.get("x") == -1
    f = LFUCache(2)
    f.put(1, 1)
    f.put(2, 2)
    assert f.get(1) == 1
    # 2 was used least often, so it goes
    f.put(3, 3)
```

```
assert f.get(2) == -1 and f.get(3) == 3
```

OFFICIAL PAGE TEXT

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items—also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n. It should support a set(key, value) method for inserting or updating items and a get(key) method for retrieving items. Can you implement a solution where both of these methods run in O(1) time?

```
Pythonclass LRUCache(n):
  set(key, value)
  get(key)
```

Examples To make room for new items, the least recently used item should be removed. An item is 'used' whenever it is set, retrieved, or updated.

```
Pythoncache = LRUCache(2) # Limit of 2 items

cache.set('user1', 'Alex')
cache.set('user2', 'Brian')
cache.set('user3', 'Chris')
```

Here user1 is empty because it was the least recently used and thus removed to make room for user3:

```
Pythoncache.get('user1') # => None
cache.get('user2') # => 'Brian'
cache.get('user3') # => 'Chris'
```

OFFICIAL PAGE TEXT

Hint 1If you're having trouble getting started, think about which data structure(s) you can use to solve this problem. Which one is best for a key-value store? Which one can help you store items in order of recency?Hint 2You'll need to be able to find and remove the least recent item in O(1) time, so some sort of ordered list is necessary. An array won't work because moving items around takes O(n) time. Could you combine two different data structures to maintain this order and also get fast lookups?Hint 3You can use a doubly linked list to store the items in the correct order. When an item is used, just move its node to the front of the list. The least recently used node will always end up at the end of the list.Hint 4To look up items without traversing the entire list, you can also add a hash table that maps from the key to the item's node.Challenge 1Whats the time complexity of your solution?This problem is challenging because of the combination of constraints such as key-value lookup, size limits, and time complexity. Ultimately, there is no single data structure that fulfills all of these needs, so we have to get creative! Our solution is to implement the cache class with a linked list and a hash table. The linked list is doubly-linked, which allows us to insert and remove items in constant time without traversing the entire list. Every time an item is set, updated, or retrieved, we just move its Node to the front of the list. When the cache reaches its size limit, we can easily make room by removing the item at the end of the list. The linked list gives us O(1) insertion time, but by itself it has O(n) lookup time because we have to traverse the entire list to find a particular item. That's not ideal since we want to be able to look up an item quickly using its key. That's where the hash table comes in: we can use it add a mapping from an item's key to its Node. Together, these data structures give us a time complexity of O(1) for insertion, deletion, and retrieval in the worst case, and a space footprint of O(n). No answers contributed by the community yet.

STUDY HERE: topic 7, Design, Streaming and Scale

HARD | DESIGN-STYLE (skippable) | Exponent | 18 of 18

Implement a function that processes a stream of interval data.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TALK **ALL TIME 8.1** **30 DAYS 0** **PRIORITY 166/228**

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 352 Data Stream as Disjoint Intervals (likely) (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/6245/implement-function-process-interval-data

QUESTION

Implement a function that processes a stream of interval data.

APPROACH

Maintain sorted disjoint intervals in a TreeMap/sorted list; on each new interval find neighbors with binary search and merge. O(log n) per update.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the function must do per interval: merge and return the current union, return total covered length, or answer point queries. Ask whether touching intervals (end equals next start) merge, whether endpoints are inclusive, and whether intervals can arrive unsorted.

BRUTE FORCE TO SAY OUT LOUD

Keep all intervals in a list and re-sort and merge on each arrival. O(n log n) per interval.

OPTIMAL IDEA

Keep disjoint intervals sorted by start. For a new interval, binary search for the first interval whose end is at least the new start and the last whose start is at most the new end; replace that slice with one merged interval. A sorted list gives O(log n) search with O(n) splice, a balanced tree gives O(log n) overall.

DRY-RUN THESE CASES

Interval inside an existing one, one that bridges two, one that touches exactly, one before everything and one after everything.

LIKELY FOLLOW-UPS

Removing a range (LeetCode 715), counting covered length incrementally, or a memory bound on the stream.

PITFALLS

INTERVIEW-DAY PLAN

Off-by-one on touching intervals. Splicing the wrong slice. Forgetting to keep the total length updated.

HOW TO THINK ABOUT IT

Keep the data in its simplest canonical form: merged intervals sorted by start with gaps between them. A new interval can only touch a contiguous block of existing ones, found with two binary searches, and that block collapses into one interval. State what 'touching' means before you code (shared endpoint, or only real overlap).

Basis: LeetCode 352, Data Stream as Disjoint Intervals (generalized from single numbers to whole intervals).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
intervals = []
def add(start, end):
    intervals.append((start, end))
def query():
    merged = []
    for s, e in sorted(intervals):
        if merged and s <= merged[-1][1]:
            last = merged[-1]
            last[1] = max(last[1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O(n log n) per query	**O(n)**

WHY IT IS SLOW

Every query re-sorts and re-merges the whole history, O(n log n) per query. Keeping the merged intervals sorted and only splicing the affected block makes an add cost O(log n) to find plus the shifted block.

OPTIMAL SOLUTION · Sorted disjoint intervals with binary search

TIME	SPACE
O(log n) to find the spot, O(n) worst-case list shift	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you keep adds fast for a very large number of intervals?

Use an ordered map (TreeMap, or a balanced tree / skip list in Python via the sortedcontainers package) keyed by start so a merge removes the swallowed intervals in amortized O(log n) instead of shifting an array. Each interval is inserted once and removed at most once, so n adds cost O(n log n) overall.

FOLLOW-UP 2 Should touching intervals such as [1, 3] and [4, 5] merge?

Agree on the semantics first. For integer points, [1, 3] and [4, 5] cover every point from 1 to 5, so they merge; use bisect_left(self.ends, start - 1). For real-valued closed intervals only overlap or a shared endpoint merges, which is what the code above does. For half-open intervals [a, b), a shared endpoint merges but nothing else does.

FOLLOW-UP 3 Answer 'is this point covered?' and 'how much is covered in total?'

The sorted disjoint intervals already support both: a point is covered if the interval with the largest start at or before it ends at or after it (one bisect), and the total length is the sum of the interval lengths (or keep it updated on each add).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement a function that processes a
# stream of interval data.
#
# Implement a function that processes a stream of interval
# data.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you keep adds fast for a very large number of
# intervals?
# 2. Should touching intervals such as [1, 3] and [4, 5]
# merge?
# 3. Answer 'is this point covered?' and 'how much is
# covered in total?'
#
# INTERVIEW NOTES
# Keep the data in its simplest canonical form: merged
# intervals sorted by start with gaps between them. A new
# interval can only touch a contiguous block of existing
# ones, found with two binary searches, and that block
# collapses into one interval. State what 'touching' means
# before you code (shared endpoint, or only real overlap).
#
# BRUTE FORCE (say it first): Keep every interval ever
# added and merge on demand
# Append each interval to a list and, whenever the union
# is needed, sort and merge from scratch. It is obviously
# right and acts as the oracle; every query pays for the
# whole history. Time O(n log n) per query. Space O(n).


from bisect import bisect_left, bisect_right
```

```
# BRUTE FORCE (say it first): Keep every interval ever
# added and merge on demand | time O(n log n) per query
class IntervalStreamBrute:
    def __init__(self):
        self.all = []

    def add(self, start, end):
        self.all.append((start, end))

    def intervals(self):
        out = []
        for s, e in sorted(self.all):
```

```
            if out and s <= out[-1][1]:
                out[-1] = (
                    out[-1][0],
                    max(out[-1][1], e),
                )
            else:
                out.append((s, e))
        return out


# BASE QUESTION (optimal): Sorted disjoint intervals with
# binary search
# Idea: Keep the merged intervals sorted by start. A new
# interval only interacts with the intervals found by two
# binary searches; replace that slice by the single merged
# interval. Touching closed intervals merge.
# FOLLOW-UP 3: Answer 'is this point covered?' and 'how
# much is covered in total?'
# Answer: The sorted disjoint intervals already support
# both: a point is covered if the interval with the
# largest start at or before it ends at or after it (one
# bisect), and the total length is the sum of the interval
# lengths (or keep it updated on each add).
class IntervalSet:
    """Keeps closed intervals merged and sorted.
    integer=True: [1, 3] and [4, 5] cover 1..5 and merge.
    integer=False: only overlap or a shared end merges."""

    def __init__(self, integer=True):
        self.starts, self.ends = [], []
        self.gap = 1 if integer else 0
        # length of [a, b] is b - a + extra
        self.extra = 1 if integer else 0
        self.covered = 0

    def length(self, start, end):
        return end - start + self.extra

    def add(self, start, end):
        if end < start:
            raise ValueError("bad interval")
        # first interval that touches the new one, and the
        # first one that is completely to its right
        i = bisect_left(self.ends, start - self.gap)
        j = bisect_right(self.starts, end + self.gap)
```

```
        for k in range(i, j):
            # the swallowed intervals leave the total ...
            self.covered -= self.length(self.starts[k],
                                        self.ends[k])
        if i < j:
            start = min(start, self.starts[i])
            end = max(end, self.ends[j - 1])
        # ... and the merged one joins it
        self.starts[i:j] = [start]
        self.ends[i:j] = [end]
        self.covered += self.length(start, end)
```

```
    # FOLLOW-UP 3: is this point covered?
    def contains(self, point):
        # the interval with the largest start <= point
        i = bisect_right(self.starts, point) - 1
        return i >= 0 and self.ends[i] >= point

    # FOLLOW-UP 3: how much is covered in total?
    def total(self):
        return self.covered

    def intervals(self):
        return list(zip(self.starts, self.ends))


# FEED a stream and print each step
def process(stream, integer=True):
    # feed the stream; yield the merged list after each
    # add
    cover = IntervalSet(integer)
    for start, end in stream:
        cover.add(start, end)
        yield cover.intervals()


if __name__ == "__main__":
    feed = [(1, 3), (8, 10), (4, 5), (6, 7)]
    steps = list(process(feed))
    assert steps[0] == [(1, 3)]
    assert steps[1] == [(1, 3), (8, 10)]
    # (4, 5) touches (1, 3) for integers; (6, 7) then
    # touches both sides, so everything becomes one
    assert steps[2] == [(1, 5), (8, 10)]
    assert steps[3] == [(1, 10)]
```

```
    real = list(process([(1, 3), (4, 5), (3, 4)], False))
    assert real[1] == [(1, 3), (4, 5)]
    assert real[2] == [(1, 5)]
    box = IntervalSet()
    for pair in [(5, 8), (1, 2), (3, 4), (20, 21)]:
        box.add(*pair)
    assert box.intervals() == [(1, 8), (20, 21)]
    assert box.total() == 10
    assert box.contains(7) and box.contains(20)
    assert not box.contains(9) and not box.contains(0)
    import random
    for _ in range(200):
        box, points = IntervalSet(), set()
        for _ in range(random.randint(1, 8)):
            a = random.randint(0, 30)
            b = a + random.randint(0, 6)
            box.add(a, b)
            points |= set(range(a, b + 1))
            assert box.total() == len(points)
        for p in range(-1, 40):
            assert box.contains(p) == (p in points)
```